Cybersecurity and Secure Programming

Compiled Revision Notes

Table of Contents

Week 1: Cybersecurity Foundations **4**
- Module Overview and Assessment . . . 4
- The 2021 HSE Ransomware Incident . . . 4
- Security Objectives and the CIA Triad . . . 5
- Threat, Vulnerability, Impact, and Risk . . . 6
- Threat Actors and Motives . . . 7
- Attack Surfaces and Attack Vectors . . . 8
- Layered Defence and Zero Trust Architecture . . . 9
- From Security Foundations to Secure Programming . . . 11

Week 2: Threat Modelling and Data Hiding **11**
- Week Overview . . . 11
- Threat Modelling Process and the Development Lifecycle . . . 12
- Shostack's Four Question Framework . . . 12
- Building a System Model . . . 13
- STRIDE Threat Categories . . . 14
- Writing Specific Threat Statements and Choosing a Response . . . 15
- Data Hiding: Steganography vs Cryptography . . . 16
- Steganography Technique Families . . . 17
- Steganalysis and Shannon Entropy . . . 18
- NTFS Alternate Data Streams (ADS) . . . 19

Week 3: NTFS Security Analysis and OWASP Top 10 **21**
- NTFS Storage Layers and Investigative Workflow . . . 21
- Logical File Size, Cluster Allocation, and File Slack . . . 22
- Comparing File Systems: FAT32, exFAT, and NTFS . . . 23
- NTFS Attribute Model and MFT Record Structure . . . 24
- Resident vs Nonresident Data and Data Runs . . . 25
- NTFS Timestamps and Metadata Comparison . . . 26
- File Deletion and Recovery Limits . . . 27
- NTFS Journals and Correlating Evidence . . . 28
- OWASP Top 10:2025 . . . 29

Week 4: Cybersecurity Signals and File Intake Validation **33**
- Establishing Identity, Integrity, and Activity . . . 33
- File Type Validation and Repeatable File-Intake Decisions . . . 33
- Cryptographic Hashes and Baseline Integrity . . . 34
- Building a Defensible Timeline and Normalising Timestamps . . . 35
- Writing Useful Application Audit Logs . . . 35
- Windows Event Viewer and Registry Artefacts . . . 36
- Assessing Combined Evidence: Observe, Interpret, Test . . . 38

Week 5: Information Disclosure, Safe Failure, and Malware **38**

Information Disclosure and Outgoing Response Design 38
The Response Path and Overbroad API Responses 39
Handling Errors Without Exposing Internals 41
Metadata, Headers, Caching, and Output Encoding 43
Username Enumeration and Timing Side-Channels 43
Secrets Management and Git Version Control History 44
Safe Logging and Failing Closed 45
Malware and Ransomware 46

Week 6: Authentication and Password Security **52**
The Portal Journey: Three Checks Across Requests 52
Password Storage and Verification Models 53
Case Study: The 2013 Adobe Breach (Troy Hunt Analysis) 55
Saving and Verifying Passwords Safely 56
Argon2id and Costly Password Hashing 57
Argon2id Parameters, Memory Budgeting, and Record Format 59

Week 7: Cookies, Sessions, and Access Control **61**
From Password Verification to Server-Side Session 61
Explicit Access Control Policy (Cedar Home Shop Case) 61
Object-Level and Function-Level Authorisation on the Server 62
Case Study: First American Financial (2019) 63
Cookie Security Attributes 64
`SameSite` Modes, Site vs Origin, and CSRF Defence 65

Week 1: Cybersecurity Foundations

Module Overview and Assessment

- **Course Assessment Structure:**

Component	Weight	Timing	Details
Quiz	25%	12th November	In-term assessment covering foundational theory and practical concepts
Written Exam	25%	15th December	Final comprehensive written examination
Project and Viva	50%	Submission 13th January + Viva 14th & 15th January	Practical cybersecurity/secure programming project with oral defence (viva)

The 2021 HSE Ransomware Incident

- **Incident Overview:** In May 2021, the Health Service Executive (HSE) in Ireland suffered a severe human-operated ransomware attack (Conti) that disrupted national healthcare services and exposed how technical weaknesses translate into operational and human consequences.
- **Incident Timeline:**

Date	Event	Significance
18 March 2021	Malicious Excel attachment opened from a phishing email	Initial foothold established on a single workstation
31 March 2021	Antivirus software detected suspicious activity	Software was configured in **monitoring mode** only and did not block execution; alerts were not acted upon
March–May 2021	Attackers compromised additional accounts and servers	Lateral movement across the internal network and theft of sensitive data
14 May 2021	Ransomware executed across the environment	Widespread encryption of servers, forcing shutdown of national IT systems
Post-Incident	Data belonging to ≈ 500 patients published online	Over 100,000 patients and staff notified by November 2022 that personal data had been accessed or copied

- **Three Stages of a Modern Intrusion:**

1. **Access:** Attackers gain an initial foothold (e.g., via a phishing attachment, stolen credentials, or an exposed service).
2. **Expansion:** Attackers move laterally through the network, escalate privileges, access additional accounts, and exfiltrate sensitive data before triggering visible disruption.
3. **Impact:** Attackers deploy the final payload (e.g., ransomware encryption) and leverage stolen data for extortion, causing operational downtime and data breaches.

- **Key Engineering Lesson:** Security failures rarely stem from a single bug in isolation. The HSE breach combined an entry vector (phishing macro), a configuration gap (antivirus in monitor-only mode), operational alert fatigue (uninvestigated detections), and weak internal segmentation (allowing lateral expansion for eight weeks).

Security Objectives and the CIA Triad

- **Core Principle:** Before selecting defensive controls, engineers must define what they are protecting and what "protection" means for a specific system.
- **The CIA Triad (NIST FIPS 199):**

Objective	Core Question	Definition	Failure Example
Confidentiality	*Who is allowed to read or obtain this data?*	Preserving authorised restrictions on information access and disclosure, including protecting personal privacy and proprietary information	Unauthorised disclosure of patient records, source code, or API keys
Integrity	*Can we trust that data and system behaviour have not been altered improperly?*	Guarding against improper information modification or destruction, including ensuring information non-repudiation and authenticity	Unauthorised alteration of bank balances, audit logs, or medication dosages
Availability	*Can authorised users rely on the system when it is needed?*	Ensuring timely and reliable access to and use of information and systems by authorised users	Ransomware encrypting hospital servers or a Denial-of-Service (DoS) outage

- **Context-Driven Priorities and Trade-Offs:**
  - Every system requires all three objectives, but their **relative priority** depends on operational context:
    - **Emergency Care System: Availability** and **Integrity** are paramount during acute treatment. A clinician needs immediate access to accurate allergy and dosage

data; an overly restrictive access workflow that delays care can endanger life, and corrupted dosage data can be fatal.
    * **Psychotherapy Notes System: Confidentiality** and **Integrity** dominate. Access is tightly restricted to specific practitioners, and brief maintenance windows are tolerable compared to unauthorised disclosure.
  - **HSE Impact Across All Three Objectives:**
    * **Confidentiality failed** when attackers exfiltrated and published patient and staff records.
    * **Integrity was cast into doubt** because administrators could not immediately verify which systems, configurations, or clinical records had been tampered with during the eight-week intrusion.
    * **Availability collapsed** when ransomware encrypted files and forced the precautionary shutdown of national healthcare IT networks, reverting hospitals to paper records.

Threat, Vulnerability, Impact, and Risk

- **Core Vocabulary:** Precision in security analysis requires distinguishing between the danger, the weakness, the consequence, and the combined exposure.

Term	Core Question	Definition
Threat	*What could happen, and who or what might cause it?*	Any circumstance or event with the potential to adversely impact organisational operations, assets, or individuals through unauthorised access, destruction, disclosure, modification, or denial of service
Vulnerability	*What weakness makes that event possible or easier?*	A flaw or weakness in system security procedures, design, implementation, or internal controls that could be exercised (accidentally triggered or intentionally exploited) and result in a security breach
Impact	*What harm follows if the event succeeds?*	The magnitude of harm (operational, financial, legal, reputational, or safety-related) that can be expected to result from the consequences of unauthorised disclosure, modification, destruction, or loss of availability

Term	Core Question	Definition
Risk	*Given how likely that event is and how serious the harm would be, how much exposure do we face?*	A measure of the extent to which an entity is threatened by a potential circumstance or event, combining the likelihood of occurrence and the resulting impact

- **Risk Relationship:**
  - In qualitative and semi-quantitative analysis, risk is modelled as a function of likelihood and impact: Risk $\approx$ Likelihood $\times$ Impact.
  - A vulnerability with no plausible threat actor or vector has low likelihood; a frequent threat targeting a system containing non-sensitive, easily replaceable test data has low impact. High risk arises when both a credible likelihood of exploitation and significant harm align.
- **Worked Example: Exposed Administration API:**
  - **Scenario:** An internal user-management API endpoint (`/api/admin/users`) is accidentally deployed to a public-facing web server without authentication checks.
  - **Vulnerability:** Missing authentication and access control on the administrative API endpoint, combined with public network exposure.
  - **Threat:** Automated scanners or external attackers discovering the endpoint and issuing requests to read user data or modify account privileges.
  - **Impact:** Unauthorised disclosure of all customer personal data (Confidentiality loss), creation of rogue administrator accounts (Integrity loss), or deletion of user accounts (Availability loss), leading to regulatory fines and reputational damage.
  - **Risk: Critical** — because automated internet-wide scanning makes discovery highly likely, exploitation requires minimal skill, and the resulting harm affects the entire user base.

Threat Actors and Motives

- **Major Threat Actor Categories:**

Actor Type	Primary Motive	Typical Capabilities and Targets
Cybercriminals	Financial gain	Organised groups using ransomware, credential theft, payment fraud, and extortion against businesses and public institutions

Actor Type	Primary Motive	Typical Capabilities and Targets
State-Linked Actors	Espionage, strategic advantage, or disruption	Well-resourced, patient groups targeting government agencies, critical infrastructure, defence, and research organisations
Insiders	Financial gain, grievance, coercion, or carelessness	Employees, contractors, or partners who already possess legitimate access and knowledge of internal systems
Opportunists	Curiosity, notoriety, or low-effort gain	Individuals using automated tools and public exploit scripts to scan for unpatched servers or default credentials

- **How Motive Shapes Observable Behaviour:**
  - Understanding *why* an attacker operates helps defenders predict *how* they behave once inside a network:

Dimension	Fast Financial Return (e.g., Ransomware Affiliate)	Long-Term Intelligence (e.g., Espionage Group)
Primary Goal	Monetise access quickly via extortion or fraud	Maintain persistent, covert access to collect sensitive information over months or years
Operational Pace	Noisy and rapid once privilege escalation succeeds	Slow, deliberate, and designed to blend into normal administrative traffic
System Impact	Intentionally disruptive (encrypting files, deleting backups, displaying ransom notes)	Avoids disruption so the victim does not discover and close the access path
Defensive Priority	Rapid containment, immutable offline backups, and blocking mass file encryption	Deep log retention, anomaly detection, credential rotation, and hunting subtle persistence mechanisms

---

Attack Surfaces and Attack Vectors

- **Distinguishing Surface from Vector:**

Concept	Definition	Analogy
Attack Surface	The complete set of points on the boundary of a system where an attacker can try to enter, cause an effect on, or extract data from that system	All doors, windows, vents, and utility pipes of a building
Attack Vector	The specific path, method, or mechanism an attacker uses to reach and exploit a point on the attack surface	Picking a specific side-door lock or climbing through an unlocked ground-floor window

- **Four Pillars of the Attack Surface:**
  1. **Interfaces:** Public web pages, REST/GraphQL APIs, file upload endpoints, network ports, administrative consoles, and command-line arguments.
  2. **Data:** User-supplied form fields, uploaded documents (PDF, DOCX, XLSX), URL query parameters, HTTP headers, cookies, and deserialised objects.
  3. **Identities:** User accounts, service accounts, API tokens, SSH keys, session cookies, OAuth grants, and third-party vendor logins.
  4. **Dependencies:** Third-party libraries (npm, PyPI, NuGet packages), base container images, operating system components, external APIs, and CI/CD build pipelines.
- **Mapping Attack Surface Points to Vectors and CIA Objectives:**

Part of the Attack Surface	Example Route (Attack Vector)	Possible CIA Property at Risk
Login Form	Credential stuffing with reused passwords	**Confidentiality** (unauthorised account access)
File Upload	Malicious content reaches unsafe server-side processing	**Integrity** or **Availability**
Public API	Changing an object identifier bypasses an ownership check	**Confidentiality** or **Integrity**
Dependency Update	A compromised package reaches the build or deployment	**Potentially all three CIA objectives**

- **Reducing the Attack Surface:** Every unused endpoint removed, unnecessary port closed, dormant admin account disabled, or redundant dependency eliminated directly shrinks the opportunities available to an attacker.

Layered Defence and Zero Trust Architecture

Layered Defence (Defence in Depth)

- **Core Principle:** No single security control is infallible. **Defence in Depth** places multiple independent controls across different layers of a system so that if one control fails or is

bypassed, subsequent layers still prevent, detect, or limit the harm.

- **Three Functional Control Layers:**

Layer	Goal	Typical Controls
Prevent	Reduce the likelihood that an attack succeeds at the boundary or during execution	Least-privilege access, strong authentication (MFA), patching, input validation, disabling unused macros/services, network segmentation
Detect and Respond	Identify malicious activity quickly and contain it before it reaches critical assets	Endpoint Detection and Response (EDR) in blocking mode, centralised security logging, anomaly alerting, incident response playbooks, isolating compromised hosts
Recover	Restore normal operations and data integrity when prevention and containment fail	Tested offline/immutable backups, disaster recovery procedures, clean system rebuild scripts, post-incident forensic review

- **The Detection-to-Response Gap:**
  - In the 2021 HSE attack, a detection control existed (antivirus flagged suspicious activity on 31 March), but because the tool was in monitoring mode and no human workflow investigated and isolated the affected machine, the detection produced **zero defensive value**. Detection without timely response is equivalent to having no detection at all.

Zero Trust Architecture (NIST SP 800-207)

- **Limitations of Perimeter Security:** Traditional "castle-and-moat" network security assumed that anything inside the corporate network firewall was trustworthy. Once an attacker bypassed the perimeter (e.g., via a single phishing email in the HSE incident), they could move laterally across internal servers with minimal restriction.
- **Core Principles of Zero Trust:**
  1. **Network Location Does Not Grant Trust:** Being connected to an internal LAN, VPN, or corporate Wi-Fi never automatically authorises a user or service.
  2. **Explicit Authentication and Authorisation per Request:** Every access request to an application, API, or database must be explicitly authenticated (*who is making the request?*) and authorised (*is this specific identity permitted to perform this action on this resource?*).
  3. **Least Privilege and Micro-Segmentation:** Users and services receive only the minimum permissions required for their current task, and resources are isolated so a compromise of one component cannot spread freely to others.
  4. **Continuous Context Re-Evaluation:** Trust is not a permanent state granted at login; session validity, device posture, and anomalous behaviour (such as sudden mass

file reads or privilege changes) are continuously re-evaluated.

From Security Foundations to Secure Programming

- **Bridging Theory to Code:** High-level security objectives and architectures are ultimately enforced (or broken) by everyday programming decisions:
  - **Every Input and Endpoint is a Trust Decision:** When a web handler reads a request parameter, parses an uploaded file, or accepts a cookie, the code is crossing a trust boundary. Treating unverified caller data as trusted enables injection, path traversal, and broken access control.
  - **Safe Defaults:** Frameworks, configurations, and application features should be secure out of the box (e.g., denying access unless a rule explicitly grants it, disabling debug endpoints in production).
  - **Server-Side Validation and Authorisation:** Client-side checks (HTML attributes, JavaScript validation, hidden UI buttons) exist only for user experience and can be bypassed trivially by modifying HTTP requests. Both input validation and permission checks must be enforced on the server for every request.
  - **Safe Output Handling and Error Management:** Encode untrusted data before rendering it into HTML or queries, and return generic public error messages while logging detailed diagnostics securely on the server.
  - **Actionable Audit Logging:** Record security-relevant events (failed logins, authorisation denials, privilege changes, file rejections) with normalised timestamps and request IDs so defenders can detect and investigate intrusions.

Week 2: Threat Modelling and Data Hiding

Week Overview

Topic Area	Core Focus	Key Analytical Question
Threat Modelling	Structured design-time analysis using DFDs, trust boundaries, and STRIDE	*What can go wrong in this architecture before we write or deploy the code?*
Steganography	Concealing the existence of a message inside an ordinary carrier file	*How can hidden payloads be embedded in media, and how do statistical signals reveal them?*
Alternate Data Streams (ADS)	NTFS file system feature allowing multiple named `$DATA` streams per file	*How can data be concealed within file system structures without changing the default stream?*

Threat Modelling Process and the Development Lifecycle

- **What is Threat Modelling?** Threat modelling is a structured engineering practice for identifying security requirements, pinpointing architectural weaknesses, evaluating potential threats, and selecting defensive controls early and continuously.
- **Security Across the Software Development Lifecycle (NIST SP 800-218):**

Lifecycle Phase	Security Activity	Output / Benefit
Requirements	Define security objectives (CIA priorities), regulatory constraints, and abuse cases	Clear security requirements alongside functional requirements
Design	Map components, data flows, and trust boundaries; apply STRIDE to identify design flaws	Architectural mitigations (e.g., adding authentication, splitting privileges) before code is written
Implementation	Apply secure coding practices (parameterised queries, server-side auth, safe file intake)	Code that enforces the design controls without introducing implementation vulnerabilities
Operation	Monitor logs, verify assumptions, patch dependencies, and update the threat model when features change	Continuous alignment between the live system and the threat model

- **Cost of Fixing Defects Across Phases (Boehm & Basili):**
  - Empirical software engineering research shows that fixing a design or security flaw after deployment can cost **10 to 100 times more** than correcting it during requirements or design. Catching a missing authorisation boundary on a whiteboard takes minutes; fixing it in production requires emergency patches, database migrations, regression testing, and potentially incident response.

Shostack's Four Question Framework

- **Framework Structure:** Adam Shostack's threat modelling methodology organises analysis around four practical questions:

Question	Focus Stage	What the Engineering Team Does
1. What are we working on?	**Scope & Model**	Build a shared representation of the system: components, actors, data stores, data flows, and trust boundaries

Question	Focus Stage	What the Engineering Team Does
2. What can go wrong?	**Threat Identification**	Systematically examine each element and boundary (e.g., using STRIDE) to write specific threat statements
3. What are we going to do about it?	**Response & Controls**	Decide whether to eliminate, mitigate, transfer, or accept each identified risk, and specify concrete engineering controls
4. Did we do a good job?	**Review & Validation**	Verify that the model matches reality, test that controls actually work, and update the model as the system evolves

---

Building a System Model

- **Data Flow Diagrams (DFDs):** Rather than modelling every line of code, a threat model uses a high-level diagram showing where data moves and where privilege or control changes.
- **Core Building Blocks of a System Model:**

Element	Symbol / Role	Examples
External Actor / Entity	People or external roles outside the system's direct control that initiate requests	Customer browser, mobile app user, system administrator
Process	Code or service that receives input, makes decisions, transforms data, or enforces rules	Web frontend server, authentication service, file processing worker
Data Store	Any location where data is kept at rest	PostgreSQL database, Redis session cache, S3 bucket, local filesystem directory
External System	Third-party services or external APIs the application communicates with	Stripe payment gateway, external OAuth provider, SMTP email relay
Data Flow	Directed arrow showing data moving from one element to another	`HTTPS POST /login`, `SQL SELECT`, `SFTP file upload`
Trust Boundary	Dashed line drawn wherever control, privilege level, or trust assumptions change	Between user browser and web server; between web server and database; between app and third-party API

- **Why Trust Boundaries Matter Most:** Every data flow crossing a trust boundary re-

quires explicit verification (authentication, authorisation, input validation, or cryptographic integrity check) because the receiving side cannot trust the sending side's assertions.

STRIDE Threat Categories

- **What is STRIDE?** Developed at Microsoft, **STRIDE** is a mnemonic checklist corresponding to six categories of threats. Each threat category represents the violation of a specific desirable security property:

Letter	Threat Category	Violated Security Property	Core Analytical Question	Concrete Example
S	**Spoofing**	**Authentication**	*Can an actor pretend to be someone or something else?*	An attacker uses a stolen session cookie or forged sender address to act as another user
T	**Tampering**	**Integrity**	*Can data in transit, data at rest, or code be modified without authorisation?*	A user alters `price=100` to `price=1` in an HTTP request or modifies an unsigned configuration file
R	**Repudiation**	**Non-Repudiation / Accountability**	*Can an actor perform an action and later plausibly deny having done it?*	An administrator deletes a sensitive record, and the system keeps no immutable audit log tying the action to their account
I	**Information Disclosure**	**Confidentiality**	*Can someone read data or internal system details they are not authorised to see?*	An API returns full internal database objects (including password hashes) or displays verbose stack traces to users

Letter	Threat Category	Violated Security Property	Core Analytical Question	Concrete Example
D	**Denial of Service**	**Availability**	*Can an actor exhaust resources or crash a component so legitimate users cannot use it?*	Uploading a massive decompression bomb or triggering expensive password hashes on thousands of concurrent requests
E	**Elevation of Privilege**	**Authorisation**	*Can an actor perform actions or access resources beyond their granted permission level?*	A standard user calls an `/admin/refund` endpoint that checks whether the user is logged in, but fails to check their role

---

Writing Specific Threat Statements and Choosing a Response

Writing Specific Threat Statements

- **Avoiding Vague Threats:** Statements like *"SQL injection"* or *"someone hacks the database"* are too vague to engineer against. A useful threat statement connects four elements (Actor + Action + Target + Consequence).
- **Example Threat Statement:**
  - *An authenticated standard user (**Actor**) modifies the `account_id` parameter in a `GET /api/invoices/{id}` request (**Action**) to the billing service (**Target**), allowing them to read another customer's confidential invoice (**Consequence**).*
- **Authentication vs Authorisation Distinction in Threat Modelling:**
  - **Authentication (Spoofing):** Verifying *who* the caller is (e.g., validating a password and session token).
  - **Authorisation (Elevation of Privilege / Information Disclosure):** Verifying *what* that authenticated caller is allowed to do or access (e.g., checking whether `session.userId === invoice.ownerId`). Many severe vulnerabilities occur after authentication has already succeeded.

Choosing and Explaining a Response

- **Four Risk Treatment Options:**

Response Strategy	Meaning	Example
Eliminate (Avoid)	Remove the feature, endpoint, or data storage that creates the risk	Stop storing plaintext credit card numbers altogether; remove an unused legacy admin endpoint
Mitigate (Reduce)	Add technical or operational controls to reduce the likelihood or impact of the threat	Enforce server-side object-level authorisation checks, rate-limiting, and parameterised queries
Transfer (Share)	Shift part of the financial or operational burden to a third party	Use an external PCI-DSS compliant payment processor (e.g., Stripe) or purchase cyber insurance (note: reputational harm cannot be transferred)
Accept (Retain)	Consciously decide to live with the risk when the cost of controls outweighs the potential harm	Accept that a public status page can be scraped, documenting the decision, its **owner**, and the **residual risk**

- **Residual Risk:** Even after applying mitigations, some risk remains (**residual risk**). Every accepted or residual risk must have a named owner and a clear rationale so it can be reviewed when system context changes.

Data Hiding: Steganography vs Cryptography

- **Two Different Goals for Protecting a Message:**

Dimension	Cryptography	Steganography
Primary Goal	Hide the **meaning (content)** of a message	Hide the **existence** of a message
Visibility to Observer	Observer clearly sees that an encrypted message or file exists, even if they cannot read it	Observer sees only an ordinary-looking carrier file (e.g., a holiday photo or audio track)
Mathematical Basis	Ciphers, cryptographic keys, and computational hardness	Redundancy and perceptual tolerance in carrier formats (images, audio, video)
Failure Mode	Compromised when the key is discovered or the cipher is broken	Compromised as soon as an analyst detects that a hidden message is present
Combined Usage	Frequently combined: a sender first **encrypts** the payload (cryptography) and then **embeds** the ciphertext into a carrier file (steganography)	

- **The Steganographic Embedding Flow:**
  1. **Message (Payload):** The secret text, key, or file to be concealed.
  2. **Carrier (Cover File):** An innocuous file (such as a 24-bit BMP/PNG image or WAV audio file) with high data redundancy.
  3. **Embedding Function (with optional Stego Key):** The algorithm that alters small, perceptually insignificant parts of the carrier to encode the message bits.
  4. **Stego Object:** The resulting file, which looks and behaves like the original carrier to casual inspection.

Steganography Technique Families

- **Major Embedding Families:**

Technique Family	How It Works	Trade-Offs (Capacity vs Robustness)
Spatial Domain (LSB Substitution)	Replaces the **Least Significant Bit (LSB)** of individual pixel colour values or audio samples with bits of the hidden message	High capacity and simple to implement, but fragile: re-saving as a lossy format, cropping, or resizing destroys the hidden payload
Spread-Spectrum	Spreads a low-energy hidden signal across a wide frequency band of the carrier using a pseudo-random noise sequence	Lower capacity than raw LSB, but much more resistant to noise and partial signal distortion
Transform Domain (DCT / Wavelets)	Modifies frequency coefficients (e.g., Discrete Cosine Transform coefficients in JPEG or Discrete Wavelet Transform values) rather than raw pixels	More resilient against compression and format manipulation; widely used in both JPEG steganography and digital watermarking

- **Worked Example: Spatial-Domain LSB Substitution:**
  - Suppose we have three 8-bit grayscale pixel values (or one RGB pixel's Red, Green, and Blue channels) with decimal values `44`, `101`, and `242`, and we want to embed the 3-bit binary sequence `101`:

Channel / Pixel	Original Decimal	Original 8-Bit Binary	Message Bit to Embed	Modified 8-Bit Binary	New Decimal	Change
Value 1	`44`	`00101100`	`1`	`00101101`	`45`	$+1$
Value 2	`101`	`01100101`	`0`	`01100100`	`100`	-1
Value 3	`242`	`11110010`	`1`	`11110011`	`243`	$+1$

  - Because changing the least significant bit alters a pixel intensity value by at most ± 1 out of 255, the human eye cannot distinguish the stego image from the original cover image.

- **Legitimate vs Malicious Uses:**
  - **Legitimate:** Digital watermarking for copyright protection, embedding medical metadata inside diagnostic imaging, or broadcast monitoring.
  - **Abuse Cases:** Malware loaders fetching second-stage shellcode hidden inside PNG images hosted on legitimate content delivery networks (CDNs), or covert data exfiltration bypassing simple content filters.

Steganalysis and Shannon Entropy

Layered Steganalysis Checks

- **What is Steganalysis?** Steganalysis is the practice of detecting the presence of hidden data inside carrier files and assessing suspicious artefacts. Analysts combine four layers of evidence:
  1. **File Properties and Structure:** Checking whether the file size is unexpectedly large for its dimensions, whether data is appended after the official end-of-file (EOF) marker, or whether container metadata has been altered.
  2. **Statistical Patterns:** Measuring byte-value distributions, LSB pair frequencies (e.g., chi-square analysis), and **Shannon entropy** across the file or within specific sections.
  3. **Visual or Audio Artefacts:** Inspecting bit-planes (extracting only the LSB of every pixel to see if a structured block or sharp boundary of random noise appears) or comparing against a known original cover file.
  4. **Context and Behaviour:** Asking *why* a background service is downloading a 15 MB uncompressed BMP icon or why a workstation is uploading hundreds of images to an external host.

Shannon Entropy

- **Definition:** Introduced by Claude Shannon, **entropy** measures the average uncertainty, unpredictability, or information density across a sequence of symbols (In file analysis, symbols are typically bytes taking 256 possible values from `0x00` to `0xFF`).

- **Shannon Entropy Formula:**

$$H(X) = -\sum_{x} p(x) \log_2 p(x)$$

  - Where $p(x)$ is the empirical probability (relative frequency) of byte value x appearing in the file or data block, and the logarithm is base 2, giving a result in **bits per byte**.

- **Interpreting the 0 to 8 Bits-per-Byte Scale:**
  - Because a byte consists of 8 bits ($2^8 = 256$ possible values), the Shannon entropy of byte data always ranges from 0 **to** 8 **bits per byte**:

Entropy Value	Meaning	Typical Data Examples
$H(X) = 0$ **bits**	Complete certainty / zero randomness ($p(x) = 1$ for a single byte value)	A file or slack region filled entirely with `0x00` null bytes or repeated `'A'` characters
$H(X) = 1$ **bit**	Two equally likely byte values ($p(x_1) = 0.5, p(x_2) = 0.5$)	A binary sequence alternating equally between only two distinct byte values
$H(X) \approx 4.0$–5.5 **bits**	Moderate structure and predictability	Plain ASCII English text, source code, HTML, or CSV files
$H(X) \approx 6.0$–7.2 **bits**	Structured binary code	Compiled native executables (`.exe`, `.dll`) containing machine instructions and padding
$H(X) \to 8$ **bits** (e.g., ≈ 7.999)	Maximum uncertainty / uniform distribution ($p(x) \approx \frac{1}{256}$ for all 256 bytes)	**Encrypted payloads**, **compressed archives** (ZIP, GZIP), compressed media (JPEG), or cryptographically random data

- **Practical Nuance on High Entropy:**
  - High entropy (close to 8, such as 7.999 in finite computer files) does **not** automatically mean a file is malicious — standard compressed formats (ZIP, PNG, JPEG) naturally exhibit high entropy. However, finding a high-entropy region inside a section that should have low entropy (e.g., inside the pixel LSB plane of a flat-colour bitmap, inside a plain-text configuration file, or in the data section of a packed executable) is a strong indicator of encryption, compression, or steganographic embedding.
- **Entropy in Triage vs Secure Programming:**
  - **In File Triage:** Analysts calculate Shannon entropy to spot packed malware, hidden encrypted blobs, or anomalous file sections.
  - **In Secure Programming:** Developers rely on the operating system's **Cryptographically Secure Pseudo-Random Number Generator (CSPRNG)** (which harvests physical and OS entropy sources) to generate unpredictable session IDs, cryptographic keys, and password salts. Note that high empirical Shannon entropy in a sample does not prove a generator is cryptographically secure (for example, a sequential counter `0x00, 0x01, ..., 0xFF` has an empirical byte entropy of exactly 8.0 bits per byte, yet is completely predictable!).

NTFS Alternate Data Streams (ADS)

- **How NTFS Organises File Data:**

- In Windows **NTFS**, a file is not a single flat blob of bytes; it is a collection of attributes stored in the **Master File Table (MFT)**.
- Every standard file has an unnamed default data attribute called the **Primary Data Stream (PDS)**, represented internally as `:$DATA`.
- NTFS also allows a single file entry to hold additional **named** `$DATA` attributes, known as **Alternate Data Streams (ADS)**, using the syntax `filename.ext:streamname` (or fully qualified as `filename.ext:streamname:$DATA`).
- Both the Primary Data Stream and any Alternate Data Streams have their attribute headers (and either their resident content or their nonresident cluster run lists) recorded inside the file's MFT record, while `$Bitmap` tracks which clusters on the volume are allocated.

- **Legitimate Uses vs Abuse Cases:**

Usage Type	Example	Explanation
Legitimate OS Feature	`brief.txt:Zone.Identifier`	Browsers and email clients write a small text stream called `Zone.Identifier` (e.g., `ZoneId=3` for Internet) to mark files downloaded from external sources so Windows can display a security warning before execution
Malicious / Evasion Abuse	`notes.txt:payload.bin` or `report.txt:secret.txt`	Attackers attach stolen data, configuration scripts, or executables to an existing benign file. Standard `dir` commands and Windows Explorer show only the size and name of the Primary Data Stream (`notes.txt`), hiding the alternate stream from casual view

- **Important Operational Limits of ADS:**

1. **An ADS Does Not Execute Itself:** Simply attaching a stream to a text file does not cause the stream to run when the text file is opened. An attacker still needs an execution trigger (such as a scheduled task, registry Run key, `wmic`, or PowerShell command) that explicitly references the stream path.
2. **File-System Bound:** ADS is a specific feature of NTFS. If a file with alternate streams is copied to a FAT32/exFAT USB drive, uploaded via a standard HTTP multipart form, or attached to an email, the non-default streams are stripped away and only the default stream is transferred.
3. **Standard Hashing Covers Only the Default Stream:** Running `Get-FileHash .\sample.txt` computes the SHA-256 hash of `sample.txt::$DATA` only; modifying or adding an alternate stream does not change the default file's hash or reported file size!

- **Inspecting and Reading Alternate Data Streams:**
  - **PowerShell Commands:**

```
# List all data streams attached to sample.txt (:$DATA and named)
Get-Item -LiteralPath .\sample.txt -Stream *

# Read the contents of a specific named stream called "review"
Get-Content -LiteralPath .\sample.txt -Stream review

# Read the Mark-of-the-Web Zone.Identifier stream
Get-Content -LiteralPath .\brief.txt -Stream Zone.Identifier
```

  - **Sysinternals Streams (`streams.exe`):** Command-line utility (`streams -s C:\TargetFolder`) that recursively scans directories for files containing non-default data streams and can strip them (`-d`).

- **Comparison: Steganography vs NTFS Alternate Data Streams:**

Property	Steganography	NTFS Alternate Data Streams (ADS)
Where Data Lives	Embedded inside the bytes of the default file content itself (e.g., pixel LSBs)	Stored in a separate named `$DATA` attribute attached to the same MFT file record
Effect on Reported File Size	May stay identical (if replacing LSBs) or grow slightly	Default file size reported by Explorer/`dir` remains **completely unchanged**
Effect on File Hash (SHA-256)	**Changes** the hash of the carrier file because its bytes are modified	**Does not change** the hash of the default stream (`::$DATA`)
Survival Across File Systems / Web	**Survives** copying to FAT32, cloud storage, email, or web uploads because it is part of the primary file bytes	**Lost** when copied to non-NTFS volumes (FAT32) or transmitted over standard network protocols
Primary Detection Method	Steganalysis (entropy anomalies, statistical LSB checks, visual bit-plane inspection)	File-system metadata inspection (`Get-Item -Stream *`, Sysinternals `streams.exe`, MFT parsing)

Week 3: NTFS Security Analysis and OWASP Top 10

NTFS Storage Layers and Investigative Workflow

- **From Physical Disk to File Artefacts:**
  - Understanding what happens when a file is created, modified, renamed, or deleted requires tracing how an operating system organises storage across layers:
    1. **Disk Image / Physical Drive:** Raw sequence of sectors on storage media.

2. **Partition Table (MBR / GPT):** Defines where each partition begins and ends on the disk.
3. **NTFS Volume:** Formatted partition starting with the **Volume Boot Sector (`$Boot`)**, which specifies bytes per sector (typically 512) and sectors per cluster (typically 8, giving a 4,096-byte cluster size) plus the starting cluster of the Master File Table.
4. **NTFS System Metadata Files:** Special reserved files (`$MFT`, `$Bitmap`, `$LogFile`, `$UsnJrnl`) that manage every file, directory, and cluster on the volume.
5. **Allocated and Unallocated Clusters:** Data blocks across the volume that either currently belong to active files (allocated) or are marked free for reuse (unallocated).

- **The Three-Step Investigative Workflow:**
  1. **Inspect:** Examine raw metadata structures (`$MFT` records, data runs, `$Bitmap` allocation flags, and journal entries).
  2. **Compare:** Cross-reference independent sources of truth (e.g., `$STANDARD_INFORMATION` vs `$FILE_NAME` timestamps, `$UsnJrnl` change records vs application logs).
  3. **Explain:** Formulate a defensible technical explanation of what occurred (and explicitly state what the evidence *cannot* prove).

Logical File Size, Cluster Allocation, and File Slack

- **Clusters as the Minimum Allocation Unit:**
  - File systems do not allocate disk space byte-by-byte; they allocate space in fixed-size blocks called **clusters** (most commonly $4,096$ bytes $= 4$ KiB on modern NTFS volumes).
  - **Logical Size (Actual File Size):** The exact number of bytes of content inside the file (e.g., $9,400$ bytes).
  - **Allocated Size (Size on Disk):** The total byte capacity of all clusters reserved for that file, which is always a multiple of the cluster size.
  - **File Slack (Slack Space):** The leftover bytes between the end of the logical file content and the end of the final allocated cluster. If the cluster was previously used by an older deleted file and not zeroed out, remnant bytes from the previous file can persist in slack space.
- **Worked Example: Calculating Clusters and Slack Space:**
  - Suppose `C:\Reports\report.csv` has a **logical size** of $9,400$ bytes on an NTFS volume with a cluster size of $4,096$ bytes:
    - **Clusters Required:** $\left\lceil \frac{9,400}{4,096} \right\rceil = \lceil 2.2949 \ldots \rceil = 3$ clusters.
    - **Allocated Size on Disk:** $3 \times 4,096$ bytes $= 12,288$ bytes.
    - **Breakdown Across Clusters:**
      - Cluster 1: $4,096$ bytes of file data (bytes 0–$4,095$)
      - Cluster 2: $4,096$ bytes of file data (bytes $4,096$–$8,191$)
      - Cluster 3: $1,208$ bytes of file data (bytes $8,192$–$9,399$) + $2,888$ **bytes of file slack**
    - **Slack Space Calculation:** File Slack $=$ Allocated Size $-$ Logical Size $= 12,288 - 9,400 = 2,888$ bytes.

Comparing File Systems: FAT32, exFAT, and NTFS

- **File System Architecture Comparison:**

Feature	FAT32	exFAT	NTFS
Primary Use Case	Small removable drives, legacy compatibility, EFI system partitions	High-capacity flash drives, SDXC cards, cross-platform removable media	Modern Windows system drives, servers, enterprise storage
Core Tracking Structure	File Allocation Table (linked list of cluster numbers) + directory entries	Allocation Bitmap + FAT chain (only used when fragmented) + stream extension entries	Master File Table (`$MFT`) where every file and metadata structure is an attribute-based record
Maximum Single File Size	4 GiB − 1 byte	16 EiB (practically unlimited)	16 TiB to 8 PiB (depending on cluster size)
Journaling & Change Tracking	None	None	Transaction log (`$LogFile`) and Update Sequence Number Change Journal (`$UsnJrnl`)
Access Control & Streams	None	None	Discretionary ACLs (permissions), encryption (EFS), and Alternate Data Streams (ADS)

- **How FAT32 Tracks and Deletes Files:**
  - **Cluster Chains:** A FAT32 directory entry stores the file's name, size, timestamps, and **starting cluster number** (e.g., Cluster 5). To find the rest of the file, the OS looks up index 5 in the File Allocation Table (FAT[5] → 9, FAT[9] → 12, FAT[12] → EOF).
  - **FAT32 Deletion (`0xE5` Marker):** When a file is deleted on FAT32:
    1. The first byte of its directory entry filename is overwritten with `0xE5` (marking the slot as deleted/available).
    2. The starting cluster number and file size remain visible in the directory entry, **but** the FAT entries (`FAT[5]`, `FAT[9]`, `FAT[12]`) are zeroed out (`0x0000`) to mark those clusters free.
  - **Conditional Recovery and Fragmentation:** If the deleted file was stored **contiguously** (e.g., clusters 5, 6, 7) and has not been overwritten, a recovery tool can read the starting cluster (5) and file size from the `0xE5` directory entry and carve the contiguous bytes. However, if the file was **fragmented** (e.g., clusters 5, 9, 12), zeroing the FAT chain destroys the links to clusters 9 and 12, making automatic reconstruction from metadata alone impossible without content-based file carving.

NTFS Attribute Model and MFT Record Structure

- **Everything is a File:** In NTFS, every user file, directory, and even NTFS's own administrative structures (`$MFT` at record 0, `$LogFile` at record 2, `$Volume` at record 3, `$Bitmap` at record 6) are stored as records inside the **Master File Table (`$MFT`)**.
- **MFT Record Layout (1,024 Bytes):**
  - Each MFT record is fixed at 1,024 **bytes** (1 KiB) and begins with the ASCII signature `FILE` (`0x46 0x49 0x4C 0x45`).
  - Inside those 1,024 bytes sit the **MFT Record Header** followed by a sequence of typed **Attributes**, terminated by an end marker (`0xFFFFFFFF`):

Component / Attribute Type ID	Attribute Name	Role and Contents
Record Header	MFT Entry Header	Magic signature (`FILE`), flags (`0x01` = in use, `0x00` = deleted/free, `0x02` = directory), sequence number, link count, offset to first attribute
`0x10`	`$STANDARD_INFORMATION`	Four core timestamps (Created, Modified, MFT Changed, Accessed), DOS file permissions (Read-Only, Hidden, System), owner/security ID
`0x30`	`$FILE_NAME`	Filename in Unicode (e.g., `report.csv`), parent directory MFT reference, namespace flag, and a separate set of four timestamps maintained by the NTFS kernel driver
`0x80`	`$DATA`	The file's content. Unnamed `0x80` is the Primary Data Stream; named `0x80` attributes are Alternate Data Streams. May be **resident** or **nonresident**

- **File Reference Numbers (`Record:Sequence`, e.g., `4128:7`):**
  - NTFS uniquely identifies a file using a 64-bit reference combining the **48-bit MFT record number** (e.g., `4128`) and a **16-bit sequence number** (e.g., `7`).
  - Every time a file is deleted and its MFT record slot (`4128`) is later reallocated to a brand-new file, NTFS increments the sequence number (`7` → `8`). This allows analysts (and the OS) to tell immediately whether a journal entry or shortcut pointing to `4128:7` refers to the same file lifecycle or an older deleted file that once occupied slot `4128`.

Resident vs Nonresident Data and Data Runs

Resident vs Nonresident Storage

- **Resident Data:**
  - If a file's content is very small (typically under ≈ 700 bytes, depending on how many other attributes exist in the $1,024$-byte MFT record), NTFS stores the entire file content **directly inside the `$DATA` attribute within the $1,024$-byte MFT record itself**.
  - **Consequence:** A resident file consumes **zero external data clusters** on the volume and therefore has **no cluster slack space** outside the MFT.
- **Nonresident Data:**
  - When a file is too large to fit inside the $1,024$-byte MFT record, NTFS marks the `$DATA` attribute header as **nonresident** (`Non-resident flag = 0x01`), allocates external clusters across the volume (updating `$Bitmap`), and stores a compact **Data Run List** (mapping pairs) inside the `$DATA` attribute instead of the raw bytes.
- **Irreversibility Rule (Nonresident Does Not Revert to Resident):**
  - **Once a file transitions from resident to nonresident, it cannot revert back to being a resident file** even if its content is later truncated or edited down to just a few bytes that could easily fit inside the MFT record. It remains nonresident until the file itself is deleted.

Virtual Cluster Numbers (VCNs), Logical Cluster Numbers (LCNs), and Data Runs

- **VCN vs LCN:**
  - **Virtual Cluster Number (VCN):** The relative cluster index within the file itself ($0, 1, 2, ...$ from the beginning of the file).
  - **Logical Cluster Number (LCN):** The absolute cluster address across the entire NTFS partition/volume ($0, 1, 2, ...$ from the start of the volume).
- **Worked Calculation: Locating Byte $8,500$ Across Nonresident Data Runs:**
  - Suppose `report.csv` ($9,400$ bytes, cluster size $= 4,096$ bytes) has a nonresident `$DATA` attribute with two fragmented data runs:
    - **Run 1:** Starts at LCN 120, length $= 2$ clusters (covers VCN 0 and VCN 1, i.e., file bytes 0 to $8,191$ at LCN 120 and LCN 121).
    - **Run 2:** Starts at LCN 900, length $= 1$ cluster (covers VCN 2, i.e., file bytes $8,192$ to $9,399$ at LCN 900).
  - **Question:** *Where on the NTFS volume is file byte offset* $8,500$ *stored?*
    1. **Find the File VCN:** VCN $= \left\lfloor \frac{8,500}{4,096} \right\rfloor = 2$.
    2. **Find the Byte Offset Within That Cluster:** Cluster Offset $= 8,500 \bmod 4,096 = 8,500 - (2 \times 4,096) = 8,500 - 8,192 = 308$ bytes.
    3. **Map VCN 2 to Volume LCN:**
       - Run 1 covers VCN 0–1 (2 clusters).
       - Run 2 starts at VCN 2, which maps to **LCN** 900.
    4. **Compute Exact Volume Byte Offset:** Volume Byte Offset $= (900 \times 4,096) + 308 = 3,686,400 + 308 = 3,686,708$ bytes.
- **Decoding Raw Hex Data Run Mapping Pairs:**
  - In the raw hex of a nonresident `$DATA` attribute, NTFS encodes run lists as compact **mapping pairs** terminated by `00` (for example, `11 02 78 21 01 0C 03 00`):
  - **First Run (`11 02 78`):**
    - Control byte `0x11`: Low nibble `1` $= 1$ byte for run length; High nibble `1` $= 1$ byte

for starting LCN offset.
    * Length byte `0x02` = 2 **clusters**.
    * Offset byte `0x78` = 120 (LCN 120).
  - **Second Run (`21 01 0C 03`):**
    * Control byte `0x21`: Low nibble `1` = 1 byte for run length; High nibble `2` = 2 bytes (little-endian signed delta) for LCN offset relative to the previous run's start.
    * Length byte `0x01` = 1 **cluster**.
    * Offset bytes `0C 03` (little-endian `0x030C`) = +780. Adding this delta to the previous run's start (LCN 120 + 780) gives **LCN** 900!
  - **Terminator (`00`):** Marks the end of the run list.

NTFS Timestamps and Metadata Comparison

- **Four NTFS Timestamp Types (MACB):**
  - NTFS stores timestamps as 64-bit integers counting 100-nanosecond intervals since **1 January 1601 00:00:00 UTC**:
    1. **Created (B / Birth):** When the file record or directory entry was created on the volume.
    2. **Modified / Last Write (M):** When the file's `$DATA` content was last altered.
    3. **MFT Changed / Entry Modified (C):** When the file's MFT metadata record itself was last updated (e.g., rename, permission change, or attribute update).
    4. **Accessed (A):** When the file was last opened/read (often disabled or deferred in modern Windows configurations to reduce disk writes).
- **`$STANDARD_INFORMATION` (SI) vs `$FILE_NAME` (FN) Timestamps:**
  - Every file has two independent sets of these four timestamps inside its MFT record:

Property	`$STANDARD_INFORMATION` (`0x10`)	`$FILE_NAME` (`0x30`)
Displayed by Explorer / `dir`?	**Yes** — standard user tools display SI timestamps	**No** — requires forensic MFT tools (e.g., `MFTECmd.exe`, Autopsy) to inspect
Modifiable by User-Mode APIs?	**Yes** — standard Windows APIs (`SetFileTime`, PowerShell `(Get-Item ...).CreationTime = ...`) directly modify SI timestamps	**No** — updated strictly by the NTFS kernel driver during file creation, rename, or move operations
Forensic Significance	Shows standard file activity, **or** what an attacker wanted the timestamp to look like if "timestomping" occurred	Acts as an independent kernel-maintained anchor; if SI Created is *earlier* than FN Created (e.g., SI claims 2021 on a file created in 2026), it strongly indicates **timestamp manipulation (timestomping)** or file copying

- **Worked Timestamp Comparison Case (`4128:7`):**
  - In the lecture's synthetic MFT record `4128:7`:
    - **SI Created (`$STANDARD_INFORMATION`):** `2026-09-01 08:00:00 UTC`
    - **FN Created (`$FILE_NAME`):** `2026-09-28 09:14:00 UTC`
    - **SI Last-Write (Modified):** `2026-09-28 09:14:02 UTC`
  - **Interpretation:** Because `$STANDARD_INFORMATION` creation (`2026-09-01`) is weeks earlier than `$FILE_NAME` creation on the volume (`2026-09-28 09:14:00 UTC`), this discrepancy is an inconsistency to investigate rather than automatic proof of malicious tampering. Plausible explanations include metadata preserved during a file copy or backup restore, specific application behaviour, or deliberate timestamp alteration (`SetFileTime`). Analysts must compare `$UsnJrnl BASIC_INFO_CHANGE` records and application logs before attributing a cause.

File Deletion and Recovery Limits

- **What Happens When a File is Deleted in NTFS:**
  1. The MFT record header flag for `4128:7` is flipped from `in use = yes` (`0x01`) to **`in use = no (0x00)`**.
  2. The file's entry is removed from its parent directory index (`$I30`).
  3. In `$Bitmap`, the bits corresponding to the file's allocated clusters (LCN $120, 121, 900$) are flipped from `1` (allocated) to **`0` (unallocated/free)**.
  4. **Crucially:** Neither the $1,024$-byte MFT record nor the data bytes inside clusters $120, 121$, and 900 are immediately zeroed out by NTFS!
- **Three Possible Recovery Outcomes:**

Outcome	Condition	What Can Be Recovered
1. Record + Content Intact	MFT slot `4128` has not been reused, and clusters $120, 121, 900$ have not been overwritten or trimmed	Full recovery: exact filename, timestamps, and complete file content reconstructed via the surviving MFT run list
2. Record Only (Metadata Survives)	MFT slot `4128` is still free, but clusters $120, 121, 900$ were reallocated to a new file or zeroed by SSD TRIM	Analysts can prove the file (`report.csv`, size, timestamps) existed and was deleted, but the file content itself is gone or corrupted
3. Content Fragments Only	MFT slot `4128` was overwritten by a new file (`4128:8`), losing the metadata and run list, while some data clusters remain unallocated	No filename or timestamps survive in the MFT; recovery relies on signature-based file carving across unallocated space

- **Three Factors That Permanently Destroy Deleted Data:**
  1. **SSD TRIM / Deallocate Commands:** On modern solid-state drives, when NTFS marks clusters free, the OS sends a `TRIM` command to the SSD controller, which zeroes

or unmaps the underlying flash blocks almost immediately.
2. **Cluster / MFT Slot Reuse:** Heavy write activity overwrites unallocated clusters and free MFT slots.
3. **Full-Disk or File-Level Encryption (BitLocker / EFS):** Without the decryption key, unallocated clusters contain only indistinguishable high-entropy ciphertext.

NTFS Journals and Correlating Evidence

- **Comparing NTFS Evidence Sources:**

Evidence Source	What It Records	Primary Strength	Primary Limitation
`$MFT`	Current state of every file/directory record (and surviving unallocated records)	Shows exact current attributes, timestamps, and cluster run lists	Shows only the *latest* state of a record, not the historical sequence of past changes
`$UsnJrnl:$J`	High-level change journal logging file creations, modifications, renames, and deletions	Captures step-by-step historical lifecycle events (such as a file being renamed and then deleted)	Does **not** record *which user or process* made the change; rings/wraps around as new changes occur
`$LogFile`	Low-level NTFS transactional metadata journal used for crash recovery	Extremely detailed record of recent metadata operations	Small circular buffer that rolls over rapidly (often within hours on an active system)
Application / OS Logs	Process execution (`4688`), user logons (`4624`), and app-level audit events	Connects actions to a **user account, process, or request ID**	May be disabled by default or lack low-level file reference numbers

- **USN Change Journal (`$UsnJrnl:$J`) Fields and Rename Sequence:**
  - Each record in the `$J` stream of `$UsnJrnl` contains:
    - **USN (Update Sequence Number):** Monotonically increasing offset identifier.
    - **Timestamp (UTC):** Exact time the file-system operation occurred.
    - **File Reference (`Record:Sequence`):** e.g., `4128:7`.
    - **Parent Directory Reference:** MFT reference of the folder containing the file.
    - **Filename:** The name of the file at that step.
    - **Reason Flags:** What happened (`FILE_CREATE`, `DATA_EXTEND`, `DATA_OVERWRITE`, `RENAME_OLD_NAME`, `RENAME_NEW_NAME`, `FILE_DELETE`, `CLOSE`).
  - **Tracing a Disguised Rename Operation (`USN 984320 to USN 984416`):**
    - If a user or script renames `report.csv` to `cache.tmp` before deleting it, the MFT record alone only shows `cache.tmp`. However, `$UsnJrnl:$J` reveals two paired entries sharing the **exact same MFT reference (`4128:7`)**:
      1. `USN 984320` | `Reason: RENAME_OLD_NAME` | `FileRef: 4128:7` | `Name: report.csv`

2. `USN 984416` | `Reason: RENAME_NEW_NAME` | `FileRef: 4128:7` | `Name: cache.tmp`
3. Later retained MFT record (`in use = no`) supports deletion of `cache.tmp` (`4128:7`).

* Because the file reference `4128:7` stays constant across the journal entries, the analyst proves that the deleted `cache.tmp` was originally `report.csv`.
* **Correlating with Application Logs:** An application log entry at `2026-09-28T09:14:02Z` reporting `export.complete`, request `r-204`, path `C:\Reports\report.csv`, and account `report-service` explains how the file was created, though the shared service account `report-service` does not by itself identify which human user triggered the export or performed the later rename.

- **Automating Metadata Review (`MFTECmd.exe`):**
  - Rather than inspecting thousands of MFT and USN records manually in a hex editor, analysts use tools like Eric Zimmerman's `MFTECmd.exe` to parse extracted `$MFT` and `$UsnJrnl:$J` (`$J`) files into structured CSV files for timeline sorting and filtering:

```
MFTECmd.exe -f '.\samples\$MFT' --csv '.\parsed'
MFTECmd.exe -f '.\samples\$J' --csv '.\parsed'
```

- **Implications for Secure Application Design:**
  - Because `unlink()` / `File.Delete()` only removes directory pointers and flips allocation bits without erasing cluster contents, applications handling temporary decrypted secrets or sensitive exports should avoid writing plaintext temporary files to disk (preferring in-memory buffers or encrypted volumes) and should write application-level audit logs that record the authenticated user alongside file operations.

OWASP Top 10:2025

Overview of the OWASP Top 10:2025 Categories

- **Purpose:** The Open Worldwide Application Security Project (OWASP) Top 10 is a community consensus document ranking the ten most critical web and software application security risk categories.

Rank	Category	Core Engineering Failure
A01:2025	**Broken Access Control**	Failing to enforce server-side checks on what authenticated (or unauthenticated) users are allowed to read or modify
A02:2025	**Security Misconfiguration**	Deploying unsafe defaults, open cloud storage buckets, permissive CORS, or verbose debug modes in production

Rank	Category	Core Engineering Failure
A03:2025	**Software Supply Chain Failures**	Relying on compromised, unverified, or tampered third-party libraries, build pipelines, or update mechanisms
A04:2025	**Cryptographic Failures**	Exposing sensitive data through missing encryption, weak algorithms, improper key management, or unsalted password hashes
A05:2025	**Injection**	Allowing untrusted input to be interpreted as executable commands or queries (SQLi, XSS, OS command injection)
A06:2025	**Insecure Design**	Architectural and business-logic flaws where security controls were never designed into the workflow
A07:2025	**Authentication Failures**	Weaknesses in verifying user identity, protecting credentials against stuffing/brute force, or invalidating sessions
A08:2025	**Software or Data Integrity Failures**	Accepting code updates, CI/CD artifacts, or serialised data objects without verifying their cryptographic integrity
A09:2025	**Security Logging and Alerting Failures**	Failing to record, protect, or monitor security-critical events, leaving intrusions invisible until impact occurs
A10:2025	**Mishandling of Exceptional Conditions**	Failing open during errors, leaving transactions in inconsistent states, or leaking internals when exceptions occur

Detailed Breakdown and Real-World Cases (A01 to A10)

- **A01:2025 — Broken Access Control:**
  - **What Goes Wrong:** Applications verify that a user is logged in (authentication) but fail to verify on the server whether that user owns the requested record (Broken Object Level Authorisation / IDOR) or possesses the required role for an action (Broken Function Level Authorisation).
  - **Real-World Case (First American Financial, 2019):** Over 800 million real-estate title and mortgage documents were exposed because document links accepted sequential document numbers in the URL without checking whether the requester was authorised

to view that specific document.
  - **Prevention:** Deny by default; enforce server-side ownership and role checks on every single request; never rely on hidden UI links or unguessable IDs alone as access control.
- **A02:2025 — Security Misconfiguration:**
  - **What Goes Wrong:** Cloud storage buckets left public, default admin credentials left unchanged, unnecessary services enabled, or development settings left active in production.
  - **Real-World Case (Toyota Connected Service, 2023):** Vehicle location and multimedia data for over 2 million customers was left accessible for years due to a misconfigured cloud database environment lacking proper access restrictions.
  - **Prevention:** Automated infrastructure-as-code templates, hardened baseline configurations, disabling debug modes in production, and continuous configuration auditing.
- **A03:2025 — Software Supply Chain Failures:**
  - **What Goes Wrong:** Modern applications assemble hundreds of open-source packages and external build tools. A compromise in an upstream dependency, package registry (typosquatting, dependency confusion), or build server compromises every downstream consumer.
  - **Real-World Case (SolarWinds Orion, 2020):** Attackers compromised the SolarWinds build environment and injected the SUNBURST backdoor into digitally signed software updates distributed to thousands of enterprise and government customers.
  - **Prevention:** Pin dependency versions with lockfiles and cryptographic hashes, maintain a Software Bill of Materials (SBOM), audit third-party packages, and harden CI/CD build pipelines.
- **A04:2025 — Cryptographic Failures:**
  - **What Goes Wrong:** Storing passwords with fast unsalted hashes (MD5, SHA-1) or reversible encryption, transmitting sensitive data over cleartext HTTP, reusing initialization vectors (IVs) or nonces, or hardcoding cryptographic keys in source code.
  - **Real-World Case (LinkedIn Breach, 2012):** Attackers stole 6.5 million (later found to be over 100 million) password hashes that had been stored as unsalted SHA-1 hashes, allowing rapid offline cracking of common passwords.
  - **Prevention:** Enforce TLS everywhere, use modern authenticated encryption (e.g., AES-GCM, ChaCha20-Poly1305) with managed keys, and hash passwords with memory-hard salted algorithms (**Argon2id**).
- **A05:2025 — Injection:**
  - **What Goes Wrong:** Untrusted user data is concatenated directly into a SQL query, OS shell command, LDAP filter, or HTML document, allowing the attacker's input to cross from the data plane into the code/control plane. Includes **SQL Injection (SQLi)** and **Cross-Site Scripting (XSS)**.
  - **Real-World Case (MOVEit Transfer, 2023):** The Cl0p ransomware group exploited a critical SQL injection vulnerability in the MOVEit managed file transfer application to steal sensitive data from thousands of organisations worldwide.
  - **Prevention:** Strictly separate code from data using **parameterised queries (prepared statements)**, context-aware output encoding, and avoiding shell interpreter invocation with user strings.
- **A06:2025 — Insecure Design:**
  - **What Goes Wrong:** Even perfectly written code cannot fix a workflow that was never designed to resist abuse. Examples include password recovery flows relying on easily researched "security questions" or high-demand checkout flows with no protection

against automated scalping bots.
  - **Real-World Case (FTC Ticket Bots Enforcement):** Automated bots bypassed weak purchasing workflows to buy thousands of event tickets in seconds because the checkout architecture lacked design-level rate limits and abuse controls.
  - **Prevention:** Perform **threat modelling** (e.g., STRIDE) during design, define explicit business-logic invariants, and build rate-limiting and anti-automation controls into critical user journeys.
- **A07:2025 — Authentication Failures:**
  - **What Goes Wrong:** Permitting weak/breached passwords, lacking rate limiting against credential stuffing, exposing session IDs in URLs, or failing to invalidate server-side sessions on logout or password change.
  - **Real-World Case (23andMe Credential Stuffing, 2023):** Attackers used lists of email/password pairs leaked from unrelated breaches to log into customer accounts that reused passwords, accessing genetic and ancestry profiles.
  - **Prevention:** Multi-Factor Authentication (MFA), checking new passwords against known breach corpora, login rate-limiting, and strict server-side session lifecycle management.
- **A08:2025 — Software or Data Integrity Failures:**
  - **What Goes Wrong:** Downloading and executing scripts or updates without verifying digital signatures/checksums, or deserialising untrusted client data into live application objects without integrity verification.
  - **Real-World Case (Codecov Bash Uploader, 2021):** Attackers modified a hosted bash script that thousands of CI pipelines downloaded and executed via `curl ... | bash` without checksum verification, exfiltrating CI environment secrets and credentials.
  - **Prevention:** Verify digital signatures and SHA-256 hashes before executing external artifacts, use Subresource Integrity (SRI) for external web scripts, and never deserialise untrusted complex object graphs.
- **A09:2025 — Security Logging and Alerting Failures:**
  - **What Goes Wrong:** Failing to log authentication failures, access control denials, or bulk data exports; storing logs only locally where an attacker can wipe them; or generating alerts that nobody monitors (as seen in the 31 March antivirus alert during the HSE incident).
  - **Real-World Case (Equifax Breach, 2017):** Attackers exploited an unpatched Apache Struts vulnerability and exfiltrated data on 147 million people over 76 days while an expired certificate prevented network inspection tools from visibility into the traffic.
  - **Prevention:** Structured, centralised, append-only security audit logs with UTC timestamps, correlation IDs, and tested alerting thresholds tied to human response workflows.
- **A10:2025 — Mishandling of Exceptional Conditions:**
  - **What Goes Wrong:** When a database times out, a third-party service fails, or an unexpected exception occurs, poorly written error handlers either **fail open** (granting access instead of denying it), leave multi-step financial or state changes half-completed (non-atomic operations), or dump internal stack traces to the caller.
  - **Real-World Case (Multi-Factor Authentication Fail-Open Flaws, 2022):** Several authentication integrations were found to catch network timeout exceptions from the MFA provider and accidentally return `true` (allowing login) instead of rejecting the request.
  - **Prevention:** Always **fail closed** (deny access unless every check explicitly succeeds),

wrap multi-step state mutations in atomic database transactions, and return safe generic errors to callers.

Week 4: Cybersecurity Signals and File Intake Validation

Establishing Identity, Integrity, and Activity

- **Three Core Analytical Questions:** When an application receives a file or when a defender investigates an incident on a host, they must rely on verifiable technical signals rather than user claims:
  1. **File Identity (Type & Structure):** *What kind of file is this really, regardless of what its filename claims?*
  2. **File Integrity:** *Are these exact bytes identical to the known baseline or expected version?*
  3. **System and Application Activity:** *When did this action occur, under which account or process, and how do different logs align on a timeline?*

File Type Validation and Repeatable File-Intake Decisions

- **Why Filename Extensions Cannot Be Trusted:**
  - A filename like `invoice.pdf` or `avatar.png` is merely a **user-supplied claim** (metadata string). Any user or attacker can rename `payload.exe` or `script.html` to `invoice.pdf`. Relying on the extension or the client-supplied HTTP `Content-Type` header allows dangerous files to bypass intake filters.
- **Three-Layer File Intake Validation:**
  1. **Layer 1 — Claimed Name and Extension Check:** Normalise the filename, strip path traversal sequences (`../`, `..\`), enforce a strict character allowlist, and check that the extension belongs to the permitted set (e.g., `.pdf`, `.png`, `.docx`).
  2. **Layer 2 — Initial Byte Check (Magic Numbers / File Signature):** Inspect the leading bytes (and, where relevant, trailer bytes) of the file content against known format signatures:

File Format	Hex Signature (Magic Bytes)	ASCII Representation
PDF Document	`25 50 44 46 2D`	`%PDF-`
PNG Image	`89 50 4E 47 0D 0A 1A 0A`	`.PNG....`
JPEG Image	`FF D8 FF`	(Binary header)
ZIP / Office Open XML (`.docx`, `.xlsx`, `.pptx`)	`50 4B 03 04`	`PK..`
Windows Portable Executable (`.exe`, `.dll`)	`4D 5A`	`MZ`

  3. **Layer 3 — Full Structural Parser Validation:**

  - Magic bytes alone are not sufficient because an attacker can prepend `%PDF-` or `89 50 4E 47` to a malicious payload (creating a polyglot or malformed file).
  - The intake pipeline must parse the entire file structure using a safe library (e.g., verifying that a PNG has valid `IHDR`, `IDAT`, and `IEND` chunks, or re-encoding the image into a fresh buffer).
  - **Important Container Nuance (`DOCX / ZIP`):** Modern Microsoft Office files (`.docx`, `.xlsx`, `.pptx`) are actually **ZIP archives** containing XML files (`[Content_Types].xml`, `word/document.xml`). Therefore, their magic bytes are `50 4B 03 04` (`PK..`). To distinguish a legitimate `.docx` from an arbitrary `.zip` containing an executable, the validator must open the ZIP structure safely (guarding against zip-bomb decompression limits) and verify the internal Office XML schema entries.

- **Repeatable File-Intake Decision Policy:**

Intake Condition	Comparison Result	Action Taken	Audit Log Requirement
Match	Claimed extension `.pdf` matches `%PDF-` signature and passes structural PDF parsing	**Accept:** Store using a newly generated random server-side filename outside the web root	Log `file_accepted` with request ID, user ID, SHA-256 hash, and validated MIME type
Mismatch	Claimed extension `.pdf` has `4D 5A` (`MZ` executable) or `50 4B 03 04` (`PK` archive) bytes, or fails structural parsing	**Reject:** Immediately discard upload; do not write to public storage; return generic validation error to caller	Log `file_rejected_mismatch` with claimed extension, detected signature, user ID, and correlation ID
Unknown / Unsupported	File signature is unrecognised or format is not on the explicit allowlist	**Reject (Fail Closed):** Deny by default	Log `file_rejected_unsupported` with request ID and user ID

Cryptographic Hashes and Baseline Integrity

- **Role of Cryptographic Hashing (SHA-256):**
  - A cryptographic hash function (such as **SHA-256**) maps an arbitrary-length sequence of bytes to a fixed-size output (256 bits = 32 bytes, displayed as 64 hexadecimal characters) with two critical properties:
    1. **One-Way (Pre-image Resistance):** Given a hash digest, it is computationally infeasible to reconstruct the original input bytes.
    2. **Collision Resistance / Avalanche Effect:** Changing even a single bit in a multi-gigabyte file produces a completely different SHA-256 hash, and it is computationally infeasible to find two distinct files that produce the exact same SHA-256 digest.

- **What a SHA-256 Baseline Comparison Proves vs Does Not Prove:**

Comparison Result	What It Proves	What It Does NOT Prove
Hash Matches Baseline	The file's primary byte stream is **byte-for-byte identical** to the baseline file at the time the baseline was recorded	Does **not** prove the file is safe if the baseline itself was already compromised; does **not** cover NTFS Alternate Data Streams (`:$DATA` only)
Hash Differs from Baseline	At least one bit of the file's content has changed since the baseline was taken	Does **not** automatically prove malicious tampering (could be a legitimate software patch, configuration edit, or version update — requires context!)

Building a Defensible Timeline and Normalising Timestamps

- **The Multi-Timezone Problem:**
  - During an investigation or across distributed microservices, evidence comes from machines configured in different local timezones or Daylight Saving Time (DST) offsets (e.g., Irish Standard Time `UTC+01:00`, Eastern Standard Time `UTC-05:00`, or UTC `Z`). Comparing raw local times directly leads to false sequences where an effect appears to happen before its cause.
- **Three Rules for Defensible Timeline Entries:**
  1. **Record the Raw Timestamp Exactly as Observed:** Preserve the original string from the log or device so evidence can be traced back to the source artefact.
  2. **Record the Explicit Offset or Timezone:** Note the UTC offset active on that source at that moment (e.g., `+01:00`).
  3. **Convert and Normalise to UTC:** Subtract a positive offset (or add a negative offset) to express every event on a single shared **UTC** axis:
     - *Example:* An event logged at `08:40 +01:00` normalises to `07:40 UTC` ($08:40 - 01:00 = 07:40$ UTC), which occurred *before* a server event logged at `08:10 UTC`.
- **Practical Caveats in Timeline Reconstruction:**
  - **Clock Skew / Drift:** If a workstation's system clock is not synchronised via NTP (Network Time Protocol) and is running 4 minutes fast, its timestamps will carry systematic uncertainty that must be documented.
  - **Archive Extraction Behaviour (e.g., ZIP):** Extracting files from a `.zip` archive often preserves the *Modified (Last Write)* timestamp stored inside the ZIP header from when the author originally created the file months ago, while the NTFS *Created* timestamp reflects when the file was extracted onto the local disk.

Writing Useful Application Audit Logs

- **Audit Logs vs Debug Logs:** Debug logs help developers troubleshoot code flow; **security audit logs** provide an accountable record of security-relevant decisions (logins, access denials, file intake rejections, privilege changes).
- **Essential Elements of an Application Audit Log Entry:**

1. **Normalised UTC Timestamp:** ISO 8601 format with millisecond precision and explicit `Z` (e.g., `2026-10-06T10:15:30.120Z`).
2. **Event Type, Outcome, and Specific Reason:** Clear machine-parseable action and result (e.g., `event: "file_intake"`, `outcome: "rejected"`, `reason: "signature_mismatch_claimed_pdf_actual_mz"`).
3. **Actor / Principal Context:** Authenticated user ID or session reference (or `"unauthenticated"` if prior to login).
4. **Correlation / Request ID:** A unique identifier generated per incoming HTTP request (e.g., `req_9f8a2b`) that links the public error reference, application audit log, and downstream service calls together.

- **What Must NEVER Be Written to Logs:**
  - **Plaintext passwords** (even failed login password attempts — users frequently mistype their real password by one character or accidentally type their password into the username field!).
  - **Session IDs, JWTs, API keys, or bearer tokens.**
  - **Full payment card numbers (PAN), CVVs, or raw sensitive personal records.**

Windows Event Viewer and Registry Artefacts

Windows Event Viewer Logs

- **Security Log vs System Log:**
  - **Security Log (`Security.evtx`):** Records audit policy events such as account logons, failed logons, privilege usage, and process creation.
  - **System Log (`System.evtx`):** Records operating system component events, driver loads, service startups/failures, and time synchronisation changes.
- **Key Windows Security Event IDs:**

Event ID	Meaning	Key Fields to Inspect
`4624`	**Successful Account Logon**	Target User Name, **Logon Type**, Source Network Address / Workstation Name, Logon ID (links activities within the same logon session)
`4625`	**Failed Account Logon**	Target User Name, Failure Reason / Sub-status code (e.g., bad password vs disabled account), Logon Type, Source IP

Event ID	Meaning	Key Fields to Inspect
4688	**A New Process Has Been Created**	Creator Process ID/Name (parent process), New Process Name, Account Name, and (if enabled via policy) **Process Command Line**

- **Distinguishing Logon Types in Events 4624 and 4625:**
  - **Logon Type 2 (Interactive):** The user logged in locally at the physical keyboard and screen (console session).
  - **Logon Type 3 (Network):** A user or service connected over the network (e.g., accessing an SMB file share \\server\share, RPC, or IIS authentication) without an interactive desktop.
- **Operational Limits of Windows Event Logs:**
  1. **Audit Policy Configuration:** Event 4688 (process creation) and its command-line logging are often **not enabled by default** on standard Windows installations unless configured via Group Policy.
  2. **Log Retention and Rollover:** Event logs have a configured maximum file size; once full, old events are overwritten unless forwarded to a central SIEM (Security Information and Event Management) server.
  3. **Administrator Tampering:** If an attacker achieves local Administrator or SYSTEM privileges, they can clear local event logs (which itself generates Event ID 1102 in the Security log) unless logs are shipped off-host in real time.

Windows Registry Persistence and Device Artefacts

- **Registry Startup Persistence (Run Keys — MITRE ATT&CK T1547.001):**
  - Attackers frequently configure malware to survive reboots by adding values to Windows Registry startup keys:
    - **User-Level:** HKCU\Software\Microsoft\Windows\CurrentVersion\Run (executes whenever that specific user logs in; does not require administrator rights to modify).
    - **Machine-Level:** HKLM\Software\Microsoft\Windows\CurrentVersion\Run (executes for every user who logs into the machine; requires administrator privileges to modify).
- **Removable Media Artefacts (USBSTOR and SetupAPI):**
  - **HKLM\SYSTEM\CurrentControlSet\Enum\USBSTOR:** Records vendor ID, product ID, version, and device serial number when a USB mass-storage device is plugged into the system.
  - **C:\Windows\INF\setupapi.dev.log:** Text log recording the first installation timestamp of plug-and-play hardware devices (such as a specific USB drive serial number).
  - **Analytical Limit:** USBSTOR and setupapi.dev.log prove that a specific USB hardware device was connected to the machine, but they do **not** by themselves prove which files

were copied to or from that drive — file movement must be corroborated using `$MFT`, `$UsnJrnl`, link files (`.lnk`), or shellbags.

Assessing Combined Evidence: Observe, Interpret, Test

- **Structured Analytical Method:**
  1. **Observe:** List the objective technical facts across sources (normalised to UTC).
  2. **Interpret:** Formulate a hypothesis connecting the observations without overstating certainty.
  3. **Test:** Check whether another log or metadata artefact confirms or contradicts the hypothesis.
- **Worked Multi-Source Correlation Example (`10:14 to 10:24 UTC`):**
  - In the lecture's fictional sequence, four observations occur within a ten-minute window:

Normalised Time (UTC)	Source Artefact	Observation	Analytical Assessment
`10:14--10:16 UTC`	Windows Security Log (`4625`)	Three failed network logons (**Logon Type 3**) for the same account	Repeated failed network authentication attempts
`10:18 UTC`	Windows Security Log (`4624`)	Successful logon for the **same account and source**	Account credentials succeeded after three failures
`10:21 UTC`	Windows Registry (`..\CurrentVersion\Run`)	New startup entry observed in `Run` key	Potential persistence mechanism added after logon
`10:24 UTC`	Windows Registry (`USBSTOR` / `SetupAPI`)	New USB storage device observed	Removable media enumerated on the host

  - **Key Investigative Takeaway:** While this `10:14–10:24 UTC` sequence strongly warrants investigation, these records **do not by themselves prove malware execution or data exfiltration**. Analysts must test alternative benign explanations and check independent sources (such as Event `4688` process creation, `$MFT`/`$UsnJrnl` file creation/access records, and application audit logs) to determine what actually ran or moved.

Week 5: Information Disclosure, Safe Failure, and Malware

Information Disclosure and Outgoing Response Design

From Input Validation to Outgoing Response Design

- **Two Directions of Trust Boundary Control:**

- In Week 4, we examined **incoming data** (validating inputs and uploaded files crossing into the server).
- In Week 5, we examine **outgoing data** (controlling what leaves the server in HTTP responses, API payloads, headers, error messages, and logs). Even if input validation is flawless, an application fails Confidentiality if its responses leak internal data or unauthorized fields.

Core Terminology

Term	Standard / Reference	Definition
Information Disclosure (Information Exposure)	**CWE-200**	Intentional or unintentional exposure of information to an actor who is not explicitly authorised to have access to that information
Excessive Data Exposure	OWASP API Security	Returning complete internal database objects or verbose JSON structures in an API response and relying on the client UI to filter out sensitive fields
Broken Object Property Level Authorisation (BOPLA)	**OWASP API3:2023**	Failing to enforce field-level (property-level) read or write permissions on an object (combining Excessive Data Exposure and Mass Assignment)
Generation of Error Message Containing Sensitive Information	**CWE-209**	Returning stack traces, database query text, file system paths, or environment variables in error responses sent to external callers

---

The Response Path and Overbroad API Responses

- **Four Stages of the Outgoing Response Path:**

Layer	What Happens	Security Question
1. Data Store	The application retrieves records and their properties from the database	*Which of these values are needed for this operation?*
2. Application	Business rules and access policy determine the result	*May this recipient receive each selected value?*

Layer	What Happens	Security Question
3. Representation	The application creates JSON, HTML, a file, or another response format	*Does the representation include only the intended properties?*
4. Delivery	Headers, caches, and the client handle the response	*Could the delivery path extend access beyond the intended recipient or use?*

- **Why Client-Side Filtering Fails (Browser DevTools):**
  - A common mistake is returning a full database user object from an API endpoint and having the frontend display only `displayName` and `course`.
  - Any user can open **Developer Tools** → **Network** → **Fetch/XHR**, trigger a request, and select **Response** (or **Headers**) to inspect the raw JSON payload before the page renders it. If a field leaves the server in the HTTP response, it has already been disclosed.
- **Worked Lecture Example: Evaluating an Overbroad Student Profile Response:**
  - Suppose a student profile page only needs to display:

```
{
  "displayName": "Mara",
  "course": "CSSP"
}
```

  - However, an overbroad generic serializer instead returns the full internal account object:

```
{
  "displayName": "Mara",
  "course": "CSSP",
  "email": "[redacted]",
  "studentNumber": "[redacted]",
  "accountStatus": "[redacted]",
  "recoveryToken": "[redacted]"
}
```

  - **Field-by-Field Security Evaluation:**

Property	Does the Profile Page Need It?	Decision to Resolve
Display name (`displayName`)	**Usually**	Confirm that this is the intended public or preferred name
Contact email (`email`)	**Sometimes**	Establish whether this recipient should see it for this specific function (e.g., authorised contact page vs public profile)

Property	Does the Profile Page Need It?	Decision to Resolve
Account recovery token (`recoveryToken`)	**No (Never)**	Exclude credential material from ordinary profile responses
Internal role / status flags (`accountStatus`, `studentNumber`)	**Usually No**	Establish whether the client genuinely needs the field or whether it is internal policy/administrative state

- **Object-Level vs Property-Level Authorisation:**
  - **Object-Level Authorisation:** *Is this caller allowed to access record #42 at all?*
  - **Property-Level Authorisation (BOPLA):** *Even if the caller is allowed to view record #42, which specific fields (properties) of #42 is this caller's role permitted to read or modify?*
- **Three Questions for Response Field Selection:**
  1. **Need:** Does the specific client screen or workflow actually require this field?
  2. **Audience:** Who is receiving this response (anonymous visitor, standard colleague, the account owner, or an administrator)?
  3. **Permission:** Is this specific caller authorised to read this specific property?
- **Allowlist Schema vs Blocklist Filtering:**
  - Always construct outgoing responses using an **explicit allowlist (Data Transfer Object / DTO projection)** that copies only approved fields:

```
// SAFE: Explicit allowlist DTO projection
function toPublicProfileDto(userRow) {
  return {
    id: userRow.id,
    displayName: userRow.displayName,
    department: userRow.department
  };
}
```

  - Avoid **blocklist** approaches (e.g., `delete userRow.passwordHash; return userRow;`), because whenever a developer later adds a new sensitive column to the database table (such as `mfaSecret` or `salary`), a blocklist automatically leaks the new column until someone remembers to exclude it.

Handling Errors Without Exposing Internals

- **Dangerous Error Patterns and What They Leak:**

Unsafe Error Type	Example Returned to Client	How It Helps an Attacker
Stack Trace	`TypeError at /srv/app/controllers/auth.js:84:12 (express 4.17.1)`	Reveals exact file system layout, programming language, framework versions (for CVE lookup), and internal logic line numbers
Database Exception	`PQError: relation "tbl_users_v2" does not exist at character 38`	Confirms SQL injection points, database engine (PostgreSQL), and exact table/column names for crafting `UNION SELECT` payloads
Account-Specific Error	`"Password incorrect for valid user alice@example.com"` vs `"No account found"`	Enables **username enumeration**, allowing attackers to harvest valid email addresses for targeted credential stuffing
Framework Debug Page	Interactive debug console dumping `process.env` and request headers	Directly exposes database connection strings, API secret keys, and session signing secrets

- **Splitting Error Handling: Public Response vs Restricted Diagnostic Record:**
  - When an unexpected server error occurs, the application should generate a short, unique **opaque error reference (correlation ID)** (e.g., `R7Q2` or `err_8f3a1c`) and split the output into two channels:

Channel	Audience	What It Contains	Example
Public HTTP Response	End user / caller browser	Generic, helpful message + HTTP status code + opaque reference ID (zero internal details)	`HTTP 500 — {"error": "We could not complete your request right now.", "reference": "R7Q2"}`
Restricted Server Log	Authorised engineers / security operations	Timestamp (UTC), reference ID `R7Q2`, full stack trace, database error code, user ID, and request route	`2026-10-13T14:22:01Z [ERROR] ref=R7Q2 userId=42 route=POST /orders err=ConnectionTimeout at db.js:41`

Metadata, Headers, Caching, and Output Encoding

Hidden Metadata, Headers, and Browser Caching

- **Where Else Information Leaks:**
  - **Document and Image Metadata (EXIF):** Uploaded photos or exported PDFs often contain GPS coordinates, author names, internal printer paths, or editing history unless stripped during server processing.
  - **Verbose Server Headers:** Headers like `Server: Apache/2.4.49 (Unix)` or `X-Powered-By: PHP/7.4.3` advertise exact patch levels to automated scanners.
  - **Shared Browser Caching of Sensitive Responses:** If an API or page returns sensitive account, medical, or financial information without cache restrictions, a shared computer's browser cache or an intermediate proxy may store a copy. Sensitive responses must include:
    `Cache-Control: no-store`
    (`no-store` instructs browsers and intermediate caches not to write the response body to persistent storage at all).

Information Disclosure vs Output Encoding

- **Distinguishing Two Different Outgoing Controls:**

Dimension	Information Disclosure Prevention	Output Encoding (Escaping)
Core Question	*Should this piece of data be sent to this caller at all?*	*How do we safely format data we intend to send so the browser does not execute it as code?*
Vulnerability Prevented	**CWE-200 / BOPLA** (Confidentiality breach)	**Cross-Site Scripting (XSS) / Injection**
Mechanism	DTO field allowlisting, property-level authorisation, generic error mapping	Converting `<`, `>`, `&`, `"`, `'` into HTML entities (`<`, `>`, etc.) or using auto-escaping templates
Key Insight	Output encoding does **not** fix information disclosure! Properly HTML-escaping a user's `passwordHash` or someone else's medical record still leaks the secret cleanly to the screen	

Username Enumeration and Timing Side-Channels

- **How Authentication Endpoints Leak Account Existence:**
  - Even if a login or password-reset form does not explicitly say *"User exists"*, an attacker can distinguish valid accounts from invalid ones through **two side-channels**:

1. **Response Wording / Status Differences:**

Scenario	Unsafe (Enumerable) Wording	Safe (Uniform) Wording
Login Failure	`"User not found"` vs `"Wrong password"`	`"Invalid email or password."` (identical status code and HTML structure for both cases)
Password Reset	`"We sent a reset link to your email"` vs `"Email address is not registered"`	`"If an account exists for that address, we have sent password reset instructions."`

2. **Response Timing Side-Channel:**
   * Suppose the server checks `SELECT * FROM users WHERE email = ?` and:
     · If **no user exists**, it returns `"Invalid email or password"` immediately in 4 **ms**.
     · If **the user exists**, it runs **Argon2id** password verification (taking 250 **ms**) before returning `"Invalid email or password"`.
   * Even though the text message is identical, an attacker measuring response latency immediately sees that 250 ms $\implies$ valid account and 4 ms $\implies$ non-existent account!
   * **Mitigation:** When a username is not found during login, execute a dummy password hash verification against a pre-generated fixed salt/hash using the exact same Argon2id parameters so both paths take $\approx$ 250 ms, and send password-reset emails asynchronously via a background queue.

Secrets Management and Git Version Control History

- **Keeping Secrets Out of Source Code:**
  – Database passwords, API keys, JWT signing secrets, and cloud credentials must never be hardcoded in source files. They should be injected at runtime via environment variables or dedicated secret managers, while only a non-sensitive template file (e.g., `.env.example` with placeholder values) is committed to Git.
- **Why Deleting a File or Adding `.gitignore` Does Not Erase Git History:**
  – Git is a snapshot-based version control system. Every commit permanently stores a snapshot of the repository's tracked files inside the hidden `.git/objects` directory.
  – **Common Developer Mistake:**
    1. Commit `a1b2c3d`: Developer accidentally commits `.env` containing `STRIPE_KEY=sk_live_998...`.
    2. Commit `e4f5g6h`: Developer deletes `.env`, adds `.env` to `.gitignore`, and commits `"Remove secret file"`.
  – **Why the Secret is Still Compromised:**
    * Adding a file to `.gitignore` only tells Git not to track *new untracked* files; it does **not** alter past commits. Anyone who clones the repository can run `git log -p` or `git show a1b2c3d:.env` and read the live key in seconds!

  - **Exposed `.git` Directories on Web Servers:** If a deployment script copies the entire repository folder (including `.git/`) into a public web root (`https://example.com/.git/`), attackers can download the commit packfiles and reconstruct the entire source code and commit history.
- **Four-Step Incident Response When a Secret Leaks into Git:**
  1. **Contain:** Restrict access to the exposed repository or deployment immediately if possible.
  2. **Revoke and Rotate Immediately (Most Critical Step):** Assume the secret was scraped by automated bots within seconds of being pushed. Revoke the old API key/password in the provider's console and issue a new one.
  3. **Investigate Access Logs:** Check the provider's audit logs to see whether the leaked credential was used by unauthorized IP addresses while it was active.
  4. **Clean History and Prevent Recurrence:** Rewrite repository history (e.g., using `git filter-repo`) to purge the blob from all commits, force-push, and add pre-commit secret scanning hooks. *Note: Cleaning Git history without rotating the key is useless once a commit has been pushed!*

Safe Logging and Failing Closed

- **Safe Audit Logging Example:**

```
// SAFE: Logs security event and correlation ID
// without logging the submitted password or token
audit({
  timestamp: new Date().toISOString(),
  event: "sign_in_failed",
  requestId: req.id,
  accountRef: user ? user.id : "unknown",
  outcome: "denied"
});
```

- **Failing Closed vs Failing Open When Permission Checks Fail:**
  - When a policy engine, database lookup, or role service throws an exception or times out during an authorisation check, the system must **fail closed (deny by default)**:

Permission Check Outcome	Safe Behaviour (Fail Closed)	Unsafe Behaviour (Fail Open — OWASP A10)
Explicitly Authorised (`true`)	Allow request to proceed	Allow request to proceed
Explicitly Denied (`false`)	Deny request (`HTTP 403 Forbidden`)	Deny request
Exception / Timeout / `null` / Unknown	**Deny request** (`HTTP 500` or `503`), log error reference, and execute zero state changes	Catch error, log a warning, and continue executing the protected handler anyway

---

Malware and Ransomware

Malware Purposes and the Role of the Compromised Computer

- **What is Malware?** Malicious software designed to infiltrate, damage, disrupt, or gain unauthorized control over a computer system against the owner's interests.
- **Three Roles a Compromised Computer Can Play:**

Role of Computer	Explanation	Example
Target	The machine or the data stored on it is the direct object of value	Stealing confidential financial spreadsheets from a CFO's laptop or encrypting a hospital's database server
Tool	The machine's compute power, bandwidth, or network identity is hijacked to attack others or generate profit	Enrolling a home router or server into a DDoS **botnet**, sending spam, or running illicit cryptocurrency miners (cryptojacking)
Both Target and Tool	The machine is harvested for local data/credentials *and* used as a stepping stone to attack connected systems	An infected workstation whose cached credentials and SMB access are used to spread ransomware across corporate file shares

Crimeware and Ransomware-as-a-Service (RaaS)

- **Specialisation in the Underground Economy:** Modern cybercrime operates as a specialised supply chain rather than lone individuals writing every tool from scratch:

Role	Function in the Ecosystem
Malware Developer (RaaS Core)	Writes, maintains, and updates the encryption payload, command-and-control (C2) panel, and extortion portal, licensing it to affiliates for a percentage of ransom payments
Initial Access Broker (Access Seller)	Specialises in breaching networks (via phishing, VPN vulnerabilities, or credential stuffing) and selling verified corporate footholds to ransomware operators
Operator / Affiliate	Purchases initial access, conducts human-operated lateral movement, exfiltrates data, disables backups, and deploys the RaaS payload
Infrastructure Provider	Supplies bulletproof hosting, domain registration, phishing kits, and cryptocurrency laundering services

Malware Categories and Mechanics

- **1. Viruses:**
  - **Mechanism:** Malware that **attaches itself to a host program or document** (such as an executable `.exe` or macro-enabled file) and relies on the host file being opened or executed to run.
  - **Two Distinct Components:**
    - **Replication Mechanism:** The code that searches for other suitable host files on the disk or shared drives and inserts copies of the virus into them.
    - **Payload:** The code that performs the malicious action (data theft, file corruption, displaying messages).
- **2. Worms:**
  - **Mechanism:** Standalone malware that **self-replicates and propagates across networks automatically** without needing to attach itself to a host program or wait for a user to open a file (e.g., exploiting unpatched network services like SMBv1 in WannaCry/NotPetya).
  - **Analytical Distinction:** Not every multi-machine infection is a network worm! In many modern ransomware incidents, attackers manually copy and execute the ransomware binary across computers using stolen domain administrator credentials, Group Policy, or PSExec. That is **human-operated administrative deployment**, whereas a true **worm** propagates autonomously via its own code.
- **3. Trojans (Trojan Horses):**
  - **Mechanism:** Malware disguised as a legitimate, desirable, or benign file (such as a cracked software installer, fake invoice tool, or system utility) to trick a user into executing it.
  - **Staged Delivery (Droppers and Downloaders):** Typically, the initial Trojan opened by the user is a small **loader/downloader** whose only job is to establish a foothold, perform basic environment checks, and fetch the heavier second-stage payload (such as an info-stealer or ransomware) from a remote server.
- **4. Backdoors vs Rootkits:**

Property	Backdoor (e.g., Remote Access Trojan / RAT)	Rootkit
Primary Function	Provides covert **remote access and command execution** bypassing normal authentication	**Conceals** the presence of malicious processes, files, network connections, and registry keys by subverting the OS
How It Works	Opens an outbound C2 channel (reverse shell, beacon) or implants an unauthorised SSH key / web shell	Hooks OS system calls (user-mode) or modifies kernel data structures and drivers (kernel-mode) so `tasklist`, `dir`, and `netstat` lie to the administrator

Property	Backdoor (e.g., Remote Access Trojan / RAT)	Rootkit
Detection Implication	Can often be spotted via live process lists, outbound network connections, or startup entries	**Cannot be reliably detected by tools running inside the compromised OS** because the OS kernel itself is filtering the results; requires **offline disk analysis** (mounting the drive on a clean forensic system) or external network monitoring

- **5. Polymorphic vs Metamorphic Malware:**
  - Attackers modify malware appearance across builds to evade simple file-hash blocklists and byte-sequence signatures:

Property	Polymorphic Malware	Metamorphic Malware
Core Mechanism	Encrypts or packs the main malicious body with a **different key on every iteration**, prepending a small variable decryption routine (stub)	**Completely rewrites its own machine instructions** on every iteration (instruction substitution, register swapping, dead-code insertion, block reordering) without needing an encrypted body
In Memory During Execution	Once the decryption stub runs in RAM, the original decrypted payload appears in memory	No fixed payload ever appears; the mutated code executes directly, performing the same logical behavior via different instruction sequences
Why Exact File Hashes Fail	Because changing the encryption key (or changing a single instruction) alters the bytes on disk, **every generated sample has a completely different SHA-256 hash** — an exact file hash only matches one specific build!	

Document-Based Malware and Social Engineering

- **Three Ways Document Files (PDF, Word, Excel) Deliver Malware:**

Vector	Mechanism	Defensive Control
Embedded Macros / Scripts	VBA macros or Excel 4.0 XLM macros inside `.docm`/`.xlsm` files execute PowerShell or download binaries when the user clicks *"Enable Content"* (as in the 2021 HSE initial access)	Block macros from files originating from the internet (`Mark-of-the-Web` Group Policy)
Reader / Parser Exploits	Specially crafted malformed font tables, image streams, or objects inside a `.pdf` or `.docx` exploit a memory-corruption vulnerability in the viewing application (Adobe Reader, Microsoft Word) without user interaction beyond opening the file	Keep document readers patched; use sandboxed/protected view modes
Embedded Phishing Links / QR Codes	The document itself contains no executable code or exploit, but displays a convincing pretext urging the user to click a link that leads to a credential-harvesting site or malware download	Email link inspection, MFA (preferably phishing-resistant WebAuthn/FIDO2), user verification workflows

- **Social Engineering and Scareware:**
  - **Pretexting:** Creating a fabricated scenario involving urgency, authority, or fear (e.g., an urgent invoice revision from a supplier, an IT password expiry notice, or a tax penalty warning) to pressure a victim into bypassing normal caution.
  - **Independent Verification:** The strongest procedural defence against pretexting is verifying unusual requests (such as bank account changes or urgent file executions) via a **separate, independently known communication channel** rather than replying to the email or calling the phone number printed inside the suspicious message.
  - **Scareware:** Deceptive pop-ups or programs that display alarming fake virus scan alerts (claiming dozens of critical infections) to trick users into paying for worthless software or installing an actual Trojan.

Malware Detection and Analysis Methods

- **Two Different Meanings of "Signature" in Security (Do Not Confuse!):**
  1. **Antivirus Detection Signature:** A pattern of bytes, strings, section hashes, or structural rules maintained by security vendors (e.g., YARA rules) to identify known malware families.
  2. **Cryptographic Digital Signature (Authenticode):** A cryptographic seal applied by a software publisher using their private key to prove **who published the binary** and that the file has **not been modified** since signing. *(Note: Attackers occasionally*

steal legitimate code-signing certificates to sign malware, so a valid publisher signature must still be paired with behavioural checks.)

- **Detection Approaches:**

Detection Method	How It Works	Strength	Weakness
Signature-Based Detection	Matches files against known byte sequences, hashes, or structural rules	Fast and very low false-positive rate on known malware	Blind to brand-new (zero-day), repacked, or polymorphic variants
Heuristic Analysis	Scans static file characteristics for suspicious traits (e.g., high-entropy packed sections, unusual API imports like `VirtualAllocEx` + `WriteProcessMemory`)	Can flag previously unseen variants that share structural suspiciousness	Higher false-positive rate on legitimate packed or obfuscated software
Behavioural Monitoring (EDR)	Watches what a running process actually *does* in real time (e.g., `winword.exe` spawning `powershell.exe`, mass rapid file renaming, clearing shadow copies via `vssadmin`)	Catches novel malware regardless of how its disk bytes were obfuscated	Must run in **blocking mode** with active alerting to stop fast encryption before damage occurs

- **Static vs Dynamic Malware Examination:**

Approach	What the Analyst Does	Advantages	Limitations
Static Analysis	Examines the binary **without executing it** (computing hashes, checking PE headers, strings, imports, entropy, and disassembling code in Ghidra/IDA)	Safe (malware cannot run) and reveals all code paths, including dormant branches	Heavily slowed down if the binary is packed, encrypted, or obfuscated

Approach	What the Analyst Does	Advantages	Limitations
Dynamic Analysis	Executes the binary inside an **isolated sandbox or virtual machine** while monitoring file system, registry, process, and network activity	Quickly reveals actual runtime behaviour, unpacked memory payloads, and C2 domains	Observes only the execution path triggered during that run; malware may detect the VM/sandbox or sleep for hours to evade observation

Ransomware Mechanics, Impact, and Recovery

- **Technical Encryption Mechanism:**
  - Modern ransomware uses **hybrid cryptography** for speed and irreversibility:
    1. It generates random symmetric keys (e.g., **AES-256** or **ChaCha20**) locally in memory to encrypt each file rapidly.
    2. It encrypts those symmetric keys using the attacker's embedded **asymmetric public key** (e.g., **RSA-2048** or **Curve25519**).
  - Because the corresponding asymmetric **private key** remains strictly on the attacker's server, the victim cannot decrypt the symmetric file keys from the disk artefacts alone.
- **Writable Network Shares:** Ransomware running under a user account does not just encrypt local `C:\` files; it enumerates every mapped drive (`Z:\`) and writable SMB network share (`\\fileserver\shared`) that the user account has permission to modify.
- **Ransomware Impact Across the CIA Triad & Double Extortion:**

Attacker Action	Primary CIA Objective Violated	Operational Consequence
Encrypting files and deleting volume shadow copies	**Availability**	Systems, databases, and clinical/business workflows are brought to a halt
Exfiltrating sensitive files before encryption ("Double Extortion")	**Confidentiality**	Even if the victim restores every server from offline backups without paying, the attacker threatens to publish stolen patient/customer data online
Modifying system configurations, tampering with records, or partial file corruption	**Integrity**	Defenders cannot immediately trust whether restored or surviving systems have been subtly altered or backdoored

- **Why Deleting the Ransomware Binary Does Not End the Incident:**

  - Simply deleting `locker.exe` from a workstation does **not** decrypt the affected files, does **not** revoke the domain credentials the attackers stole during the expansion phase, and does **not** remove secondary backdoors, scheduled tasks, or web shells planted on other servers.

- **Backups vs Synchronised Cloud Copies:**
  - **Synchronised Folders (OneDrive, Dropbox, live RAID mirrors) are NOT offline backups:** If ransomware encrypts files on a workstation and the sync client automatically replicates those encrypted changes to the cloud folder (or if an attacker uses compromised admin credentials to wipe the cloud console), the copies are destroyed alongside the originals.
  - True recovery requires **offline, immutable, or air-gapped backups** whose management credentials are completely separate from the corporate Active Directory domain, combined with **regular restoration testing** to verify that backups actually boot and restore within the required recovery time.

- **Summary of Defensive Controls Against Ransomware:**

Control	Primary Layer	How It Reduces Risk
Phishing-Resistant MFA & Disabling Office Macros	Prevent (Initial Access)	Blocks credential theft and prevents malicious document attachments from executing
Least Privilege & Read-Only Network Shares	Prevent / Contain (Expansion)	Limits which network folders and servers a single compromised workstation can encrypt
EDR in Blocking Mode + Monitored Alerts	Detect and Respond	Halts suspicious process trees (`excel.exe -> powershell.exe`) and isolates hosts before mass encryption
Immutable / Offline Tested Backups	Recover	Enables restoration of systems and data without paying a ransom

---

Week 6: Authentication and Password Security

The Portal Journey: Three Checks Across Requests

- **Tracking Identity and Permission Across a Web Session:**
  - HTTP is stateless by itself. When a user signs into a portal and navigates between pages, the server must perform three distinct checks across the journey:

Journey Step	Server Security Check	Core Question Answered
1. Login (`POST /login`)	**Authentication**	*Does this submitted password verify against the stored credential record for this account?* (If yes, create a server-side session and issue a session cookie)
2. Next Request (`GET /dashboard`)	**Session Validation**	*Does the incoming request carry a valid, unexpired session identifier that maps to an active authenticated account on the server?*
3. Resource Request (`GET /results/7201`)	**Authorisation (Access Control)**	*Is the authenticated account associated with this session permitted to view or modify resource `7201`?*

- **Why IP Addresses Are Unreliable for Tracking Identity:**
  - A server must **never** use the client's source IP address as a substitute for a session token:
    1. **Shared IPs (NAT / Corporate Proxies):** Hundreds of students on a university Wi-Fi network or employees behind a corporate NAT gateway appear to the server with the **exact same public IP address**. Identifying sessions by IP would mix their accounts together!
    2. **Changing IPs (Mobile Roaming):** A single legitimate user on a smartphone switching from Wi-Fi to 5G mobile data changes IP addresses mid-journey.

Password Storage and Verification Models

- **Comparing Three Ways Servers Have Stored Passwords:**

Storage Approach	How Login Verification Works	What Happens When the `users` Table Leaks in a Database Breach
Plaintext	Server compares `input === stored_password`	**Immediate total compromise:** Attackers read every user's exact password directly with zero effort

Storage Approach	How Login Verification Works	What Happens When the `users` Table Leaks in a Database Breach
Reversible Encryption (e.g., AES / 3DES)	Server decrypts `stored_ciphertext` using a server key (or encrypts input with the same key) and compares	**Catastrophic compromise:** If the encryption key leaks alongside the app config, all passwords decrypt instantly; even **without** the key, deterministic encryption leaks identical passwords and enables structural analysis (as in the 2013 Adobe breach)
Salted Password Hash (`Argon2id`)	Server reads the user's unique stored `salt` and parameters, runs `Argon2id(input, salt, params)`, and compares the output digest in constant time	**Strongest protection:** Cannot be reversed mathematically; unique per-user salts prevent precomputed rainbow tables and hide whether two users share a password; memory-hard cost slows offline guessing to a crawl

- **Distinguishing Encryption, Fast Cryptographic Hashing, and Password Hashing:**

Operation	Reversible?	Speed Goal	Primary Use Case
Encryption (e.g., AES-GCM)	**Yes** (with key)	**Fast** (hardware-accelerated)	Protecting data at rest or in transit that the system **must later decrypt and read** (e.g., medical notes, files, HTTPS traffic)
Fast Cryptographic Hash (e.g., SHA-256)	**No** (One-way)	**Extremely Fast** (billions of hashes/sec on GPUs)	Verifying **file integrity**, digital signatures, and Git commit content where inputs are large or high-entropy
Password Hashing (e.g., Argon2id)	**No** (One-way)	**Intentionally Slow & Memory-Hard**	Storing **human-chosen passwords** (which have low entropy and must never be recovered in plaintext)

Case Study: The 2013 Adobe Breach (Troy Hunt Analysis)

- **What Happened:**
  - In October 2013 (analysed by Troy Hunt on 12 November 2013), attackers breached Adobe and stole a database dataset containing **152,989,508 account records**, of which **109,305,888 included a plaintext password hint** (and the top 100 most frequent hints alone appeared across **7,365,869 records**).
  - Instead of hashing passwords with a salted one-way password hash, Adobe had **encrypted** every password using **3DES in Electronic Codebook (ECB) mode** with a **single shared encryption key** and **zero per-user salt/IV**, alongside storing user-written password hints in cleartext!
- **How Deterministic Encryption + Plaintext Hints Defeated Password Protection Without the Key:**
  - Even without knowing Adobe's secret 3DES key, Troy Hunt's analysis demonstrated how deterministic encryption and readable hints exposed millions of accounts:
    1. **Equal Passwords Produced Identical Ciphertexts:** Because every password was encrypted deterministically under the same key without a unique salt or IV, any two users who chose the exact same password had the **exact same stored encrypted string (ciphertext)** in the database.
    2. **Ciphertext Clusters Under a Single Hint (`"color"`):**
       * Among accounts whose password hint was literally `"color"`, grouping records by their exact encrypted password string revealed large clusters of users sharing the same password:
         · **Encrypted value 1: 3,011 accounts** storing that exact identical ciphertext.
         · **Encrypted value 2: 1,733 accounts** storing that exact identical ciphertext.
         · **Encrypted value 3: 1,042 accounts** storing that exact identical ciphertext.
       * *Analytical nuance:* The ciphertext frequency chart counts accounts sharing an encrypted string rather than directly proving the plaintext word on its own, and knowing one group's word does not reveal the 3DES encryption key — but identifying the password for just *one* member of a group immediately compromises all other accounts in that matching ciphertext group!
    3. **Categories Among the Top 100 Most Frequent Distinct Password-Hint Strings:**
       * Troy Hunt classified the **100 most frequent distinct hint strings** (which occurred across **7,365,869 records**). Note that the denominator is **100 distinct hint strings** (not percentages of all accounts — e.g., `"Family name: 24"` means 24 of the top 100 distinct hint strings fell into that category):

Hint Category (out of Top 100 Distinct Hint Strings)	Count of Hint Labels in Top 100	Examples / Meaning
Family name	`24`	Surnames or relative names
Password repeat	`14`	Hints that literally repeat or state the password
Other	`12`	Miscellaneous phrases (`usual`, etc.)

Hint Category (out of Top 100 Distinct Hint Strings)	Count of Hint Labels in Top 100	Examples / Meaning
Animal	12	Pet classifications (`dog`, `cat`, etc.)
Numbers	10	Number sequences (`123456`, etc.)
Contact data	7	Phone numbers, addresses, postcodes
Discoverable name	6	First names or public names
Birthday or date	6	`birthday`, birth years, anniversaries
Social media	4	Platform names or handles
Sport	3	Teams or sports
Love	2	Endearments or partner references

4. **Cross-Breach Verification (Adobe Hints Paired with Sony Plaintext Passwords):**
   * By correlating email addresses across breaches to compare Adobe plaintext hints with Sony plaintext passwords, Hunt showed how hints reveal either the password family, a direct riddle answer, or personal data:

Adobe Plaintext Hint	Paired Sony Plaintext Password	Information the Hint Supplies
`fruit`	`grapefruit`	A likely family of words
`animal that says neigh`	`horses`	A clue with a widely known answer
`name birthyear`	`mike1953`	An interpretation that may reveal personal information

   * *(Note: These are cross-breach pairs rather than directly decrypted Adobe ciphertexts, so they do not prove every paired account used the exact same password on both sites, but they illustrate why readable hints undermine password secrecy).*

- **Two Core Engineering Lessons from Adobe (2013):**
  1. **Storage Lesson:** Never encrypt passwords (especially deterministically without per-record randomness). Always use a **salted, memory-hard one-way password hash (`Argon2id`)** so identical passwords never produce identical stored records.
  2. **Recovery Lesson:** Never store or display **password hints** or static "security questions". Changing an encryption algorithm alone would not fix the information leaked by readable hints; recovery must use a proper **password reset flow** rather than retaining a clue to the existing password.

Saving and Verifying Passwords Safely

- **Registration / Password Change Flow:**

1. The client sends the plaintext password to the server over **HTTPS (TLS)** (never hash solely on the client as a substitute for server hashing, because an unsalted client hash just becomes a static password equivalent).
2. The server validates password policy (e.g., minimum length, checking against known breached password lists).
3. The password hashing library generates a fresh, unique 16**-byte (**128**-bit) random salt** from the operating system's **CSPRNG**.
4. The library runs **Argon2id** combining the password, the random salt, and the configured cost parameters (`m`, `t`, `p`).
5. The server stores the formatted **password-hash record string** (which embeds the algorithm ID, version, parameters, salt, and output hash) in the `users` table and immediately discards the plaintext password from memory.

- **Three-Step Login `verify()` Flow:**
  1. **Fetch Stored Record:** Look up the user account record from the database and read its stored Argon2id string (extracting the exact **salt** and **parameters** `m`, `t`, `p` that were used when that password was saved).
  2. **Recompute Digest:** Run `Argon2id(submitted_password, stored_salt, stored_params)` to compute a candidate digest.
  3. **Constant-Time Comparison:** Compare the candidate digest against the stored hash digest using a **constant-time byte comparison** (so timing side-channels cannot leak how many leading bytes of the hash matched).

Argon2id and Costly Password Hashing

- **Why Fast Hashes (SHA-256) Are Unsafe for Passwords:**
  - Modern GPUs and ASICs contain thousands of parallel arithmetic cores with tiny per-core memory. Because `SHA-256` requires almost zero memory (just a few CPU registers), an attacker with consumer GPUs can test **tens of billions of `SHA-256` password guesses per second** offline.
- **Comparing `SHA-256` vs `Argon2id`:**

Property	`SHA-256`	`Argon2id` (Winner of Password Hashing Competition, RFC 9106)
Design Goal	Maximum throughput per byte	Configurable **computational and memory cost** per hash
Built-in Salt & Parameter Format	None (raw digest only)	Yes — standard encoded string stores version, `m`, `t`, `p`, salt, and hash
Memory Requirement	Negligible (< 1 KiB)	Configurable large memory block (e.g., 19 **MiB to** 64 **MiB per hash**)

Property	`SHA-256`	`Argon2id` (Winner of Password Hashing Competition, RFC 9106)
Resistance to GPU / ASIC Cracking	**Very Weak** — massively parallelizable on GPUs	**Strong** — requiring 64 MiB of fast RAM per hash starves thousands of GPU cores of memory bandwidth (memory-hardness)

- **Comparing the Three Argon2 Variants (RFC 9106):**

Variant	Memory Indexing Mode	Primary Strength	Primary Weakness / Recommendation
`Argon2d`	**Data-dependent** memory addressing (uses current password/state values to pick which RAM block to read next)	Strongest resistance against GPU/ASIC time-memory trade-off (TMTO) cracking	Vulnerable to **cache-timing side-channel attacks** on shared hardware
`Argon2i`	**Data-independent** memory addressing (memory access pattern does not depend on the password)	Immune to cache-timing side-channel attacks	Weaker against GPU time-memory trade-off attacks than `Argon2d`
`Argon2id`	**Hybrid:** First half of the first pass uses **data-independent (`Argon2i`)** addressing; all subsequent passes/slices use **data-dependent (`Argon2d`)** addressing	Combines side-channel protection on the initial pass with strong GPU/TMTO resistance on subsequent passes	**Recommended default standard (OWASP & RFC 9106)** for all password hashing

- **How Unique Per-Account Salts Stop Precomputation and Equality Leaks:**
  - Suppose both **Account A** and **Account B** choose the exact same weak password `"blue"`:

Account	Submitted Password	Library-Generated Salt	Stored `Argon2id` Result
Account A	`"blue"`	`Random salt A` (`c2FsdA...`)	`Hash A` (`$argon2id$...$K9x8m...`)

Account	Submitted Password	Library-Generated Salt	Stored `Argon2id` Result
Account B	`"blue"`	`Random salt B` (`ZGlmZg...`)	`Hash B` (`$argon2id$...$7Qp2v...`)

  - **Two Critical Benefits of the Unique Salt:**
    1. **No Equality Leak:** Looking at the database, the two hashes look completely unrelated; an attacker cannot tell that Account A and Account B share the same password.
    2. **No Precomputation / Multi-Target Shortcut:** An attacker cannot precompute a single rainbow table or test `"blue"` once against all 100 million users simultaneously. They are forced to run the full 64 MiB Argon2id calculation separately for Account A's salt and again for Account B's salt!
  - *Note:* A salt is **not** a secret key — it is stored in plaintext right inside the database hash record. It does not prevent an attacker from guessing passwords against *one specific single account* if that user chose `"123456"`; stopping single-account guessing is the job of **password length/complexity rules** and **Argon2id's memory/time cost parameters**.

Argon2id Parameters, Memory Budgeting, and Record Format

Tuning `m`, `t`, and `p` and Server Memory Budgeting

- **The Three Cost Parameters of Argon2id:**

Parameter	Name	Unit / Meaning	Example Setting
`m`	**Memory Cost**	Amount of RAM allocated per hash operation, specified in **kibibytes (KiB, where** $1,024$ **KiB** = 1 **MiB)**	`m=65536` ($65,536$ KiB = 64 MiB per hash)
`t`	**Time Cost (Iterations)**	Number of passes (iterations) over the allocated memory block	`t=3` (3 passes)
`p`	**Parallelism (Lanes)**	Number of independent parallel memory lanes/threads used per single hash computation	`p=4` (4 lanes, so each lane manages $\frac{64 \text{ MiB}}{4} = 16$ MiB)

- **Worked Calculation: Concurrent Verification Memory Budgeting:**

- Because `Argon2id` intentionally consumes a large block of RAM for each active password check, engineers must budget server RAM carefully to prevent **Out-Of-Memory (OOM) crashes / Denial of Service (DoS)**:
- **Scenario:** An authentication server is configured with `m=65536` (64 MiB per hash) and handles 20 **concurrent login verifications** at peak load:
  * **Memory per Verification:** $\frac{65{,}536 \text{ KiB}}{1{,}024} = 64$ MiB.
  * **Total Peak Hashing RAM:** 20×64 MiB $= 1{,}280$ MiB $= 1.25$ GiB.
- **Protecting the Login Endpoint from DoS:** If an attacker floods `/login` with 500 simultaneous requests and the server starts all 500 Argon2id hashes at once (500×64 MiB $= 32$ GiB), the server will crash! Therefore, production servers combine:
  1. **Concurrency Semaphore / Worker Pool Limit:** Capping the maximum number of simultaneous Argon2id calculations allowed in flight (e.g., max 20 concurrent verifications; excess requests wait briefly in a bounded queue or receive `HTTP 429 / 503`).
  2. **Rate Limiting:** Limiting login attempts per IP and per account.
  3. **Benchmarking on Target Hardware:** Tuning `m`, `t`, and `p` on the actual production CPU/container size so a single verification takes roughly 150–500 **ms** without exhausting server memory.

Reading a Stored Argon2id Record

- **Standard Modular Crypt Format String:**
  - Example stored record: `$argon2id$v=19$m=65536,t=3,p=4$<salt>$<hash>`

Segment	Value in Example	Meaning
Algorithm Identifier	`argon2id`	Specifies the hybrid Argon2id algorithm
Algorithm Version	`v=19`	Argon2 specification version `0x13` (19 in decimal, not the year 2019)
Cost Parameters	`m=65536,t=3,p=4`	64 MiB RAM (65,536 KiB), 3 passes, 4 parallel lanes
Salt and Hash	`<salt>$<hash>`	Base64-encoded random per-password salt and expected output digest

  - **Seamless Parameter Upgrades:** Because every row stores its own `m`, `t`, and `p` parameters, when hardware improves next year and you increase the config for new hashes to `m=131072` (128 MiB), existing users can still log in seamlessly! On a successful login, the server inspects the user's stored record, sees it used the older `m=65536`, and transparently re-hashes the validated password with `m=131072` and saves the upgraded record.

Week 7: Cookies, Sessions, and Access Control

From Password Verification to Server-Side Session

- **Five-Step Session Lifecycle:**
  1. **Login Verification:** User submits credentials to `POST /login`; server verifies the password against the stored `Argon2id` record.
  2. **Session Creation on Server:** The server generates a fresh, unpredictable 128**-bit+ random Session ID** using a CSPRNG (regenerating any pre-login session ID to prevent **session fixation**) and stores a session record in server storage (`sessionId -> { accountId: 101, role: "customer", expiresAt: ... }`).
  3. **Issuing the Cookie (`Set-Cookie`):** The server sends the HTTP response with a `Set-Cookie` header containing the opaque session ID and defensive cookie attributes:
     `Set-Cookie: __Host-session=<unpredictable-id>; Path=/; Secure; HttpOnly; SameSite=Lax; Max-Age=1800`
  4. **Subsequent Requests (`Cookie`):** On future requests to the site, the browser automatically attaches the cookie in the request header:
     `Cookie: __Host-session=<unpredictable-id>`
     (Notice that the browser sends only the `name=value` pair in the `Cookie` request header; the attributes `Secure`, `HttpOnly`, `SameSite`, and `Max-Age` are instructions from the server to the browser during `Set-Cookie` and are not echoed back).
  5. **Termination / Expiry:** When the user clicks **Log Out** (`POST /logout`), the server **deletes/invalidates the session record in the server-side store** (merely clearing the cookie in the browser without deleting the server session leaves the token usable if it was previously captured!) and sends `Max-Age=0` to clear the browser cookie.
- **A Session ID is a Credential:** Anyone who possesses an active session ID can impersonate that user for the lifetime of the session without knowing their password or passing MFA. Therefore, session IDs must be protected in transit, in the browser, and in server logs with the same rigour as passwords.

Explicit Access Control Policy (Cedar Home Shop Case)

- **Why Access Policies Must Be Explicit:** Developers cannot write correct authorisation checks unless the application's rules for **who can do what to which object** are stated unambiguously.
- **Cedar Home Shop Specification Example:**
  - Consider an online store (**Cedar Home**) with two customer accounts (**Lina**, who owns purchase `#7201`, and **Omar**, who owns purchase `#7202`), a **Support Staff** account, and **Unauthenticated** visitors:

Caller Identity	View Purchase `#7201` (Lina's Order)	View Purchase `#7202` (Omar's Order)	Issue Refund (`POST /purchases/:id/refund`)
No Valid Session (Anonymous)	**Deny** (`401 Unauthorized`)	**Deny** (`401 Unauthorized`)	**Deny** (`401 Unauthorized`)

Caller Identity	View Purchase #7201 (Lina's Order)	View Purchase #7202 (Omar's Order)	Issue Refund (POST /purchases/:id/refund)
Lina (role: "customer", owns #7201)	**Allow** (200 OK)	**Deny** (403 Forbidden)	**Deny** (403 Forbidden)
Omar (role: "customer", owns #7202)	**Deny** (403 Forbidden)	**Allow** (200 OK)	**Deny** (403 Forbidden)
Support Staff (role: "support")	**Allow** (200 OK)	**Allow** (200 OK)	**Allow** (200 OK)

Object-Level and Function-Level Authorisation on the Server

Object-Level Authorisation (Preventing BOLA / IDOR)

- **What is BOLA / IDOR?**
  - **Broken Object Level Authorisation (BOLA)**, also known as **Insecure Direct Object Reference (IDOR)**, occurs when an application takes a record identifier from the request (such as `/purchases/7201` or `?invoiceId=7201`), verifies that the caller is logged in, and fetches the record from the database **without verifying that the logged-in account is authorised to access that specific object**.
  - This is a **horizontal access control failure**: Customer Omar (`role: "customer"`) accesses a resource belonging to peer Customer Lina (`role: "customer"`) at the same privilege level.
- **Server-Side Object-Level Policy Implementation:**

```
function canViewPurchase(account, purchase) {
  if (!account || !purchase) {
    return false; // Fail closed if either record is missing
  }
  return account.id === purchase.ownerId ||
         account.role === "support";
}
```

- **Why Every Detail of `canViewPurchase` Matters:**
  1. **Null / Missing Guard (`!account || !purchase`):** Fails closed (`false`) if the user is not authenticated or if the purchase object is undefined (preventing `undefined === undefined` from accidentally evaluating to `true`!).
  2. **Strict Equality (`===`):** In JavaScript, `===` checks both value and type without unsafe type coercion.
  3. **Explicit Ownership or Authorised Role Check:** Grants access **only** if the authenticated session's `account.id` matches `purchase.ownerId`, or if the authenticated account holds the `"support"` role.

4. **Why Input Validation Does Not Prevent BOLA:** Validating that `:id` in `/purchases/7201` is a positive integer (`7201`) proves the input is well-formed syntax, but `7201` is a completely valid integer whether Lina or Omar sends it! Only comparing the **authenticated session identity** against the **resource owner** stops BOLA.

Function-Level Authorisation (Preventing BFLA)

- **What is BFLA?**
  - **Broken Function Level Authorisation (BFLA)** occurs when an administrative or privileged action endpoint (such as `POST /purchases/7201/refund` or `DELETE /api/users/42`) fails to verify the caller's role on the server, assuming that because the *"Issue Refund"* button is hidden in the customer UI, customers won't call the endpoint.
  - This is a **vertical access control failure**: a standard customer invokes a function reserved for a higher privilege role (`"support"` or `"admin"`).
- **Server-Side Function-Level Policy Implementation:**

```
function canIssueRefund(account) {
  if (!account) {
    return false; // Fail closed when unauthenticated
  }
  return account.role === "support";
}
```

Case Study: First American Financial (2019)

- **What Happened:**
  - In May 2019, First American Financial Corporation exposed over **800 million real-estate title, mortgage, tax, and wire-transaction documents** dating back to 2003 through its **EaglePro** document-sharing web application.
  - When a customer received an email link to view their closing document (e.g., `.../?docid=123456789`), the server returned the document **without requiring authentication or checking whether the viewer was associated with that transaction**.
  - Changing the sequential `docid` parameter in the URL by $+1$ or -1 allowed anyone on the internet to view any other customer's sensitive financial documents (classic **BOLA / IDOR** under **OWASP A01: Broken Access Control**).
- **Why Random IDs (UUIDs) Alone Do Not Replace Authorisation:**
  - A common question is: *"What if we replace sequential IDs (7201, 7202) with random 128-bit UUIDs?"*
  - Using unpredictable UUIDs makes blind enumeration harder (a useful defence-in-depth measure), **but it does NOT replace server-side authorisation checks**:
    - URLs containing UUIDs are still stored in browser history, leaked in `Referer` headers, forwarded in emails, captured in proxy/access logs, and shared across team chats.
    - If a resource contains private account data, the server must still verify `canViewPurchase(account, purchase)` on every request regardless of how the ID is formatted.

Cookie Security Attributes

- **Controlling How the Browser Stores and Sends the Session Cookie:**

Cookie Attribute / Prefix	Example	What It Instructs the Browser to Do	Threat Mitigated
`Secure`	`Secure`	Only transmit this cookie over encrypted **HTTPS** connections; never send it over plaintext HTTP	Network eavesdropping / session hijacking on unencrypted links
`HttpOnly`	`HttpOnly`	Forbid client-side JavaScript (`document.cookie`) from reading or modifying this cookie; only the browser's HTTP engine can attach it to requests	Prevents an attacker's injected XSS script from reading the raw session ID string via `document.cookie` and exfiltrating it
`SameSite`	`SameSite=Lax`	Control whether the browser attaches the cookie on **cross-site requests** initiated from external websites	**Cross-Site Request Forgery (CSRF)**
`Max-Age`	`Max-Age=1800`	Expire and delete the cookie automatically after $1,800$ seconds (30 minutes)	Limits exposure window if a device is left unattended
`Path`	`Path=/`	Scope the cookie to all paths on the host (note: `Path=/app` is *not* a security boundary against malicious code running on another path of the same origin because same-origin scripts can open an iframe to `/app`!)	Functional scoping

Cookie Attribute / Prefix	Example	What It Instructs the Browser to Do	Threat Mitigated
`__Host-` Prefix	`__Host-sid=...`	Browser enforces **three strict rules**: cookie **must** have `Secure`, **must** have `Path=/`, and **must NOT** have a `Domain` attribute (locking the cookie strictly to the exact host that set it)	**Cookie tossing / subdomain overwrite attacks** (prevents a compromised subdomain `blog.example.com` from overwriting the session cookie of `shop.example.com`)

- **Critical Limitation — Why `HttpOnly` Does Not Cure Cross-Site Scripting (XSS):**
  - While `HttpOnly` prevents a malicious JavaScript payload from reading `document.cookie` to steal the session ID string, **an XSS script running inside the victim's browser can still call `fetch("/purchases/7201/refund", { method: "POST" })` directly!**
  - Because the `fetch()` call originates from the victim's own browser on the same origin, the browser automatically attaches the `HttpOnly` session cookie to that request. Thus, `HttpOnly` stops raw token theft, but it does **not** stop an XSS attacker from performing any action the logged-in user can perform while the tab is open.

`SameSite` Modes, Site vs Origin, and CSRF Defence

Understanding Cross-Site Request Forgery (CSRF)

- **How CSRF Works:**
  - Suppose Lina is logged into `https://cedarhome.example` (her browser holds an active session cookie).
  - In another browser tab, she visits an attacker's page (`https://coupon-blog.attacker.test`), which contains a hidden auto-submitting HTML `<form action="https://cedarhome.example/account/email" method="POST">`.
  - If the browser automatically attaches Lina's `cedarhome.example` session cookie to that cross-site `POST` request, the Cedar Home server sees a valid session cookie and changes Lina's email address without her knowledge!

`SameSite` Cookie Modes: `Strict`, `Lax`, and `None; Secure`

- **Comparison Across Request Contexts:**

Request Scenario	`SameSite=Strict`	`SameSite=Lax` (Modern Browser Default)	`SameSite=None; Secure`
Same-Site Request (Navigating or clicking buttons inside `cedarhome.example`)	**Sent**	**Sent**	**Sent**
Cross-Site Top-Level `GET` Navigation (Clicking an `<a href="https://cedarhome.example/orders">` link inside an email or external site)	**NOT Sent** (User appears logged out on arrival!)	**Sent** (User arrives still logged in — smooth UX)	**Sent**
Cross-Site Subresource / State-Changing Request (External `<form method="POST">`, cross-site `fetch()`, `<img>`, or `<iframe>`)	**NOT Sent**	**NOT Sent**	**Sent** (Requires explicit CSRF tokens / origin validation!)

Distinguishing `Origin` vs `Site` (Why It Matters for `SameSite`)

- **Web Security Defines Two Different Boundaries:**

Concept	Definition	Components Compared	Example Compared to `https://shop.example.com:443`
Origin (Same-Origin Policy)	Exact match of **Scheme + Full Hostname + Port**	`scheme`, full `hostname`, `port`	`https://promo.example.com:443` is a **different origin** (subdomain differs)
Site (`SameSite` Cookies)	Match of **Scheme + Registrable Domain** (`eTLD+1`, i.e., effective top-level domain like `.com` or `.co.uk` plus one label before it)	`scheme`, `eTLD+1` (`example.com`)	`https://promo.example.com` is the **SAME SITE** because both share `https://` and the registrable domain `example.com`!

- **Security Consequence of `SameSite` Using "Site" Rather Than "Origin":**
  - Because `https://old-marketing-blog.example.com` and `https://banking.example.com` share the same registrable domain (`example.com`), browser cookies consider requests between them to be **same-site**!
  - Therefore, if any subdomain under `example.com` is vulnerable to XSS or takeover, `SameSite=Strict` or `SameSite=Lax` alone **will not block** cross-subdomain requests from sending `banking.example.com`'s cookies.

Choosing Cookie Settings Around the User Journey and Layered CSRF Defence

- **Selecting the Right Cookie Configuration by Application Type:**

Application Journey	Recommended `SameSite` Mode	Rationale
1. Ordinary E-Commerce / Web Portal (e.g., Cedar Home Shop)	**`SameSite=Lax`** (with `__Host-`, `Secure`, `HttpOnly`)	Users clicking a link to a product or order from an email or search engine arrive logged in (top-level `GET`), while cross-site `POST` forms from external websites do not carry the session cookie
2. High-Sensitivity / Tighter Isolation App (e.g., Internal Admin Console or Banking Transfer Portal)	**`SameSite=Strict`** (or separate re-authentication prompt)	Even a top-level link click from an external site will not carry the session cookie, ensuring no cross-site navigation immediately lands inside an authenticated state
3. Embedded Cross-Site Service (e.g., Third-Party Payment Widget or SSO `<iframe>`)	**`SameSite=None; Secure`**	Required only when the service is intentionally embedded inside third-party domains; must be paired with strict anti-CSRF tokens, `Origin` header checks, and `frame-ancestors` Content Security Policy rules

- **Three Golden Rules for Complete CSRF Defence:**
  1. **Never Perform State Changes on `GET` Requests:** `SameSite=Lax` allows cross-site top-level `GET` navigations to carry cookies. If your application deletes an order or issues a refund via `GET /orders/7201/delete`, an attacker only needs to trick a user into clicking a normal link (or opening a popup window) to trigger the state change! All state-changing actions must require `POST`, `PUT`, `PATCH`, or `DELETE`.
  2. **Use `SameSite=Lax` (or `Strict`) on Session Cookies:** Blocks standard cross-site `POST` form submissions from external domains by default.
  3. **Enforce Anti-CSRF Tokens and `Origin`/`Referer` Header Verification:** For sensitive state-changing actions (and to protect against same-site subdomain attacks where

`SameSite` does not trigger), validate a cryptographically random, session-bound **CSRF token** and verify that the request's `Origin` header matches the exact expected application origin.